

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Phạm Tất Thành

**GIẢI PHÁP TĂNG TÍNH HIỆU QUẢ CỦA CÁC CÔNG CỤ
KIỂM THỬ BẢO MẬT ĐỘNG CHO ỨNG DỤNG WEB**

KHÓA LUẬN TỐT NGHIỆP
Ngành: KHOA HỌC MÁY TÍNH

Cán bộ hướng dẫn: TS. Lê Khánh Trình

Cán bộ đồng hướng dẫn: TS. Nguyễn Thị Thủy

HÀ NỘI - 2025

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và sâu sắc nhất đến ban giám hiệu Trường Đại học Công nghệ - Đại học Quốc gia Hà Nội và thầy cô Bộ môn Công nghệ phần mềm của Khoa Công nghệ thông tin đã tạo cho tôi một môi trường và điều kiện thuận lợi cho tôi trong suốt quá trình nghiên cứu và hoàn thành khóa luận.

Đặc biệt, tôi xin bày tỏ lòng biết ơn sâu sắc tới thầy TS. Lê Khánh Trình, thầy đã tận tình chỉ dẫn, định hướng, truyền đạt kiến thức và góp ý chi tiết để tôi có thể hoàn thành khóa luận tốt nghiệp này.

Cuối cùng, tôi xin gửi lời cảm ơn đến gia đình và bạn bè, những người đã luôn bên cạnh động viên và ủng hộ tôi trong suốt quá trình nghiên cứu và hoàn thiện khóa luận.

Lời cam đoan

Tôi là Phạm Tất Thành, sinh viên lớp QH-2022-I/CQ-I-CS4 khóa K67 theo học ngành Khoa học máy tính tại trường Đại học Công Nghệ - Đại học Quốc gia Hà Nội. Tôi xin cam đoan đề tài khóa luận tốt nghiệp “Giải pháp tăng tính hiệu quả của các công cụ kiểm thử bảo mật động cho ứng dụng Web” là kết quả nghiên cứu do bản thân tôi thực hiện trong thời gian nghiên cứu tại trường. Các nội dung nghiên cứu, kết quả trong khóa luận này là xác thực.

Các thông tin sử dụng trong khóa luận này là có cơ sở và không có nội dung nào sao chép từ các tài liệu mà không ghi rõ trích dẫn tham khảo. Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày 29 tháng 08 năm 2025

Sinh viên

Phạm Tất Thành

Tóm tắt

Khóa luận này tập trung vào việc nghiên cứu và đề xuất các giải pháp nhằm nâng cao hiệu quả của các công cụ kiểm thử bảo mật động (Dynamic Application Security Testing - DAST) cho ứng dụng Web. Trong bối cảnh chuyển đổi số, có rất nhiều sản phẩm ứng dụng Web được ra đời thì kéo theo đó các cuộc tấn công mạng ngày càng phức tạp, việc phát hiện sớm và khắc phục các lỗ hổng bảo mật là vô cùng quan trọng. Các công cụ kiểm thử bảo mật động, với khả năng mô phỏng tấn công từ bên ngoài ứng dụng mà không cần biết chi tiết mã nguồn của ứng dụng, đóng vai trò quan trọng trong quy trình phát triển và đảm bảo chất lượng phần mềm. Tuy nhiên, chúng vẫn còn tồn tại nhiều hạn chế như tỷ lệ sai sót cao, khó khăn trong việc kiểm thử các ứng dụng phức tạp sử dụng công nghệ Web hiện đại và thiếu khả năng bao phủ các lỗ hổng bảo mật nguy hiểm.

Khóa luận này đề xuất một số giải pháp nhằm nâng cao khả năng bao phủ trong kiểm thử bảo mật động, bao gồm việc mở rộng khả năng phát hiện lỗ hổng bảo mật của công cụ kiểm thử ZAP đối với ứng dụng Web; đề xuất quy trình kiểm thử bảo mật dành cho ứng dụng Web nhằm tăng khả năng bao phủ các lỗ hổng và khai thác tối đa hiệu quả của công cụ; đồng thời kết hợp ZAP với các công cụ khác dựa trên mô hình ngôn ngữ lớn nhằm tăng cường khả năng tự động phát hiện lỗ hổng bảo mật và rút ngắn thời gian tiếp cận với các lỗ hổng phổ biến trong ứng dụng Web. Sau khi triển khai các giải pháp đề xuất trong nghiên cứu, công cụ ZAP được xây dựng site-map đầy đủ và được bổ sung thêm tập luật kiểm thử, từ đó giúp nâng cao khả năng nhận diện các lỗ hổng bảo mật phổ biến, đặc biệt là những lỗ hổng thuộc nhóm TOP 25 CWE và TOP 10 OWASP.

Từ khóa: Kiểm thử bảo mật động (Dynamic Application Security Testing - DAST), Mô hình ngôn ngữ lớn (Large Language Model - LLMs), Ứng dụng Web, Công cụ kiểm thử ZAP.

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Mục lục iii

Danh sách hình vẽ v

Danh sách bảng vi

Danh mục các từ viết tắt vii

Chương 1 Đặt vấn đề 1

1.1 Giới thiệu bài toán 1

1.2 Những thách thức lớn trong bài toán 2

1.3 Các nghiên cứu liên quan 3

1.4 Mục tiêu và đóng góp của khóa luận 5

Chương 2 Kiến thức cơ sở 7

2.1 Kiểm thử bảo mật cho ứng dụng web 7

2.1.1 Top 10 OWASP và Top 25 CWE cho ứng dụng Web 7

2.1.2 Các phương pháp để kiểm thử bảo mật trên ứng dụng web . . . 12

2.2 Công cụ kiểm thử bảo mật động 14

2.3 Mô hình ngôn ngữ lớn 15

2.3.1 Cơ chế hoạt động của mô hình ngôn ngữ lớn 15

2.3.2 Các công cụ làm việc với mô hình ngôn ngữ lớn 16

**Chương 3 Phương pháp cải thiện độ hiệu quả trong kiểm thử bảo
mật trên ứng dụng web 17**

3.1	Hướng tiếp cận	17
3.2	Cách thức thực hiện	19
3.2.1	Phương pháp xây dựng site-map đầy đủ cho ứng dụng Web SPA	19
3.2.2	Phương pháp phát triển tập luật cho công cụ ZAP nhằm tăng cường khả năng phát hiện lỗ hổng	23
3.3	Tổng kết	26
Chương 4 Thực nghiệm và đánh giá		27
4.1	Mục tiêu thực nghiệm	27
4.2	Quy trình thực nghiệm	30
4.2.1	Thông số đánh giá	30
4.2.2	Luật kiểm thử bổ sung	32
4.2.3	Kiểm thử bảo mật trước khi cải tiến với khả năng mặc định của công cụ	36
4.2.4	Kiểm thử bảo mật sau khi áp dụng các phương pháp kiểm thử cải tiến	39
4.3	Tổng kết	45
Chương 5 Kết luận		46
Kết luận		46
Tài liệu tham khảo		48

Danh sách hình vẽ

3.1	Luồng kiểm thử bảo mật với phương pháp xây dựng site-map thủ công.	20
3.2	Luồng kiểm thử và xây dựng site-map tự động.	21
3.3	Luồng kiểm thử với tài liệu OpenAPI.	23
3.4	Luồng hoạt động của ZAP với luật kiểm thử tùy chỉnh.	24
3.5	Kiến trúc RAG được thiết kế cho quá trình sinh luật kiểm thử ZAP.	25
4.1	Minh hoạ luồng hoạt động của luật tùy chỉnh cho lỗ hổng LDAP Injection.	32
4.2	Minh hoạ luồng hoạt động của luật tùy chỉnh CWE-200.	33
4.3	Minh hoạ luồng hoạt động của luật tùy chỉnh CWE-352.	34
4.4	Minh hoạ luồng hoạt động của luật tùy chỉnh CWE-93.	36

Danh sách bảng

4.1	Số lượng ca kiểm thử theo từng loại lỗ hổng trong Benchmark v1.2 .	27
4.2	Các lỗ hổng bảo mật xuất hiện trong OWASP Juice Shop	28
4.3	Kết quả đánh giá theo từng loại lỗ hổng của công cụ ZAP trên trang Web OWASP Benchmark	37
4.4	Khả năng phát hiện các CWE trong OWASP Juice Shop bằng công cụ ZAP	38
4.5	Kết quả phát hiện lỗ hổng trên Broken Crystal sau khi bổ sung site-map	40
4.6	Kết quả phát hiện lỗ hổng trên OWASP Juice Shop sau khi bổ sung site-map	41
4.7	Kết quả đánh giá phát hiện lỗ hổng LDAP Injection và XPath Injection của phương pháp trên OWASP Benchmark	41
4.8	Kết quả phát hiện lỗ hổng trên OWASP Juice Shop sau khi bổ sung luật	43
4.9	Kết quả phát hiện lỗ hổng trên Broken Crystal sau khi bổ sung mã kiểm thử	44

Danh mục các từ viết tắt

STT	Từ viết tắt	Cụm từ đầy đủ	Cụm từ tiếng Việt
1	DAST	Dynamic Application Security Testing	Kiểm thử bảo mật động
2	SAST	Static Application Security Testing	Kiểm thử bảo mật tĩnh
3	IAST	Interactive Application Security Testing	Kiểm thử bảo mật tương tác
4	LLM	Large Language Model	Mô hình ngôn ngữ lớn
5	API	Application Programming Interface	Giao diện lập trình ứng dụng
7	RAG	Retrieval-Augmented Generation	Tạo sinh dựa trên truy xuất tăng cường
8	CWE	Common Weakness Enumeration	Các điểm yếu chung
9	LDAP	Lightweight Directory Access Protocol	Giao thức truy cập thư mục nhẹ
10	XPath	XML Path Language	Ngôn ngữ đường dẫn XML
11	WSTG	Web Security Testing Guide	Hướng dẫn cách kiểm tra bảo mật với Web
12	UI	User Interface	Giao diện người dùng
13	URL	Uniform Resource Locator	Địa chỉ tài nguyên thống nhất
14	SPA	Single Page Application	Ứng dụng đơn trang
15	FP	False Positives	Dương tính giả
16	FN	False Negatives	Âm tính giả
17	AI	Artificial Intelligence	Trí tuệ nhân tạo
18	Site-map	Site-map	Bản đồ ứng dụng

Chương 1

Đặt vấn đề

1.1 Giới thiệu bài toán

Ứng dụng Web hiện nay đã trở thành nền tảng quan trọng cho nhiều hoạt động trong giai đoạn chuyển đổi số, từ kinh doanh đến dịch vụ công, đồng thời đặt ra yêu cầu ngày càng cao về an toàn thông tin và bảo vệ hệ thống trước các mối đe dọa mạng. Sự phát triển nhanh chóng và khả năng tiếp cận rộng rãi của các ứng dụng Web kéo theo nguy cơ an ninh mạng ngày càng gia tăng. Với vai trò là cổng giao tiếp chính giữa người dùng và hệ thống, ứng dụng Web thường xuyên là mục tiêu của các cuộc tấn công mạng, khai thác những lỗ hổng phổ biến như SQL Injection, Cross-Site Scripting (XSS) hay Broken Authentication. Hậu quả của các cuộc tấn công thành công không chỉ dừng lại ở gián đoạn dịch vụ mà còn bao gồm rò rỉ dữ liệu nhạy cảm, thiệt hại tài chính và suy giảm uy tín của tổ chức. Theo báo cáo Verizon Data Breach Investigations Report (DBIR) [1], các sự cố liên quan đến ứng dụng Web tiếp tục chiếm tỷ trọng lớn trong các vụ vi phạm dữ liệu, cho thấy nhu cầu cấp thiết trong việc kiểm thử và phát hiện sớm các rủi ro bảo mật ứng dụng.

Để đối phó với các mối đe dọa, nhiều phương pháp kiểm thử bảo mật đã được nghiên cứu và áp dụng xuyên suốt vòng đời phát triển phần mềm, tiêu biểu gồm: Kiểm thử bảo mật tĩnh (SAST) [2], Kiểm thử bảo mật động (DAST) [3] và Kiểm thử bảo mật tương tác (IAST) [4]. Các phương pháp này không tồn tại một cách độc lập mà mang tính bổ trợ lẫn nhau, hướng tới mục tiêu chung là phát hiện và giảm thiểu rủi ro bảo mật ở các giai đoạn khác nhau của quá trình phát triển và vận hành ứng dụng. Trên thực tế, việc kết hợp nhiều phương pháp kiểm thử là xu hướng nhằm nâng cao mức độ bao phủ và hiệu quả bảo mật tổng thể cho hệ thống. DAST giữ vai trò đặc thù trong nhóm các phương pháp kiểm thử bảo mật khi tập trung đánh giá mức độ an toàn của ứng dụng Web trong trạng thái vận hành, từ góc nhìn của tác nhân tấn công bên ngoài. Phương pháp này cho phép kiểm chứng trực tiếp khả năng khai thác các lỗ hổng trong môi trường triển khai, phản ánh

sát hơn các rủi ro bảo mật mà hệ thống phải đối mặt khi đưa vào sử dụng. Do đó, DAST được sử dụng rộng rãi trong các giai đoạn kiểm thử, triển khai và bảo trì.

Xuất phát từ vai trò thực tiễn, khóa luận lựa chọn DAST làm trọng tâm nghiên cứu, không nhằm phủ nhận vai trò của các phương pháp khác, mà nhằm tập trung phân tích sâu một khía cạnh cụ thể trong hệ sinh thái kiểm thử bảo mật ứng dụng. Bên cạnh những ưu điểm, DAST vẫn tồn tại nhiều hạn chế, chẳng hạn như tỷ lệ cảnh báo sai, khó khăn trong việc kiểm thử trên Web SPA, cũng như khả năng bao phủ chưa đầy đủ đối với một số nhóm lỗ hổng nguy hiểm. Việc nhận diện và phân tích các hạn chế này giúp đề xuất các hướng cải tiến, góp phần nâng cao hiệu quả của các công cụ DAST và đảm bảo chất lượng bảo mật cho các ứng dụng Web.

1.2 Những thách thức lớn trong bài toán

Mặc dù đóng vai trò quan trọng, các công cụ DAST vẫn phải đối mặt với nhiều thách thức mang tính đặc thù, làm giảm hiệu quả phát hiện lỗ hổng và gây tốn kém nguồn lực, cần nghiên cứu và cải tiến liên tục. Các công cụ DAST hiện có vẫn tồn tại những điểm yếu cơ bản ảnh hưởng đến khả năng hoạt động tối ưu. Một trong những vấn đề lớn nhất là tỷ lệ sai sót cao, bao gồm cả cảnh báo sai (FP) và bỏ sót lỗ hổng (FN). Cảnh báo sai (FP) làm lãng phí thời gian của chuyên gia bảo mật, giảm hiệu suất; còn bỏ sót lỗ hổng (FN) tiềm ẩn rủi ro nghiêm trọng khi lỗ hổng thực sự bị bỏ qua. Thêm vào đó, các công cụ DAST gặp nhiều hạn chế khi kiểm thử các ứng dụng Web SPA vốn phụ thuộc mạnh vào JavaScript, AJAX và các framework như React, Angular hay Vue.js. Do đặc thù tải nội dung động và cơ chế điều hướng động dựa trên sự kiện, phần lớn URL và trạng thái quan trọng của ứng dụng không được thu thập đầy đủ bởi thuật toán khám phá của công cụ. Những hạn chế này khiến các công cụ DAST không được hình thành site-map đầy đủ và giảm phạm vi cần kiểm thử. Hệ quả là khả năng bao phủ bị giảm và nguy cơ bỏ sót lỗ hổng tăng cao.

Một hạn chế quan trọng khác là *tập luật quét bảo mật sẵn có của các công cụ DAST vẫn chưa đủ mạnh* để phát hiện toàn diện các lỗ hổng phổ biến trong nhóm Top 25 CWE và Top 10 OWASP. Nhiều loại lỗ hổng nguy hiểm chỉ được nhận diện ở mức hạn chế hoặc không được phát hiện do thiếu các mẫu kiểm thử và kỹ thuật khai thác phù hợp. Điều này cho thấy sự cần thiết phải mở rộng và cải tiến hệ

thống luật kiểm thử nhằm nâng cao khả năng phát hiện lỗ hổng và tăng hiệu quả của quá trình kiểm thử bảo mật động.

Cuối cùng, *khả năng xử lý xác thực và phân quyền của các công cụ DAST vẫn còn hạn chế*. Việc duy trì trạng thái đăng nhập và kiểm thử các chức năng yêu cầu quyền hạn cụ thể thường gặp nhiều khó khăn, đòi hỏi cấu hình thủ công phức tạp và làm tăng nguy cơ bỏ sót lỗ hổng trong các khu vực được bảo vệ. Điều này đặc biệt gây trở ngại cho những người mới tiếp cận kiểm thử bảo mật, đồng thời tác động tiêu cực đến kết quả và hiệu suất chung của quá trình kiểm thử. Do đó, cần có một cách tiếp cận mới nhằm khắc phục những hạn chế đã được đề cập, rút ngắn thời gian phát hiện các lỗ hổng phổ biến trong ứng dụng Web đồng thời đảm bảo được độ chính xác trong kiểm thử.

1.3 Các nghiên cứu liên quan

Việc đảm bảo chất lượng bảo mật cho ứng dụng Web đòi hỏi sự kết hợp của nhiều phương pháp khác nhau, mỗi phương pháp nhắm đến các loại lỗ hổng và giai đoạn phát triển phần mềm riêng biệt. SAST phân tích mã nguồn để phát hiện lỗ hổng sớm nhưng có thể thiếu chính xác và không thấy lỗ hổng xuất hiện trong thời gian ứng dụng chạy [2]. DAST tấn công ứng dụng đang chạy để tìm lỗ hổng trong môi trường vận hành, không cần mã nguồn nhưng đồng thời cũng gặp hạn chế về độ chính xác và khả năng bao phủ các lỗ hổng quan trọng [5]. IAST kết hợp SAST và DAST bằng cách đặt tác nhân vào ứng dụng đang chạy để phân tích luồng dữ liệu và mã nguồn, cho độ chính xác cao hơn [4]. Cuối cùng, kiểm thử thâm nhập thủ công [6] do chuyên gia thực hiện vẫn là phương pháp quan trọng để phát hiện các lỗ hổng logic hoặc những vấn đề phức tạp mà công cụ tự động khó nhận diện. Tuy nhiên, cách tiếp cận này đòi hỏi chi phí lớn và khó mở rộng.

Nhiều nghiên cứu đã nỗ lực cải thiện hiệu quả DAST, bởi đây là kỹ thuật kiểm thử quan trọng, giúp đánh giá ứng dụng từ góc nhìn của kẻ tấn công. Một hướng quan trọng là tích hợp DAST vào quy trình CI/CD (DevSecOps) [7] để tự động hóa và phát hiện lỗ hổng sớm, mục tiêu của hướng đi này là rút ngắn thời gian quét và giảm cảnh báo giả. Các nghiên cứu khác đề xuất cách tiếp cận lai, bằng cách kết hợp DAST, SAST và IAST [8] hoặc các nguồn thông tin khác như log của ứng dụng, đưa log vào phân tích và kết hợp giúp tăng độ chính xác và phát hiện

sớm các hành vi tấn công [9]. Một lĩnh vực khác đầy hứa hẹn là áp dụng các mô hình học máy và trí tuệ nhân tạo để phát hiện các hành vi khai thác thông tin không đúng từ hệ thống, cải thiện khả năng nhận diện các loại lỗ hổng, giảm cảnh báo giả, tăng cường khả năng khám phá ứng dụng và tự động tạo kịch bản kiểm thử [10]. Tuy nhiên, việc triển khai mô hình học máy và ứng dụng trí tuệ nhân tạo vẫn đối mặt với thách thức về dữ liệu huấn luyện và tài nguyên tính toán. Các công cụ DAST phổ biến như OWASP ZAP [11] và Burp Suite [12] đã và đang không ngừng được cải tiến dựa trên các nghiên cứu này.

Mặc dù đã có nhiều nghiên cứu nhằm cải tiến DAST, vẫn tồn tại những khoảng trống cần được giải quyết để nâng cao khả năng bao phủ các lỗ hổng bảo mật của các công cụ kiểm thử. Sự phức tạp trong logic nghiệp vụ cùng với các cơ chế kết xuất nội dung động, chỉ kết xuất một phần cần thiết và không kết xuất lại toàn bộ trang, nhiều URL chỉ xuất hiện khi có sự kiện, khiến thuật toán khám phá của công cụ gặp nhiều hạn chế trong việc thu thập và xây dựng đầy đủ site-map. Bên cạnh đó, khả năng phát hiện lỗ hổng bảo mật của các công cụ DAST vẫn còn giới hạn ở nhiều loại lỗ hổng cũng như ở các cách thể hiện khác nhau của từng lỗ hổng. Chẳng hạn như với mã CSRF, được dùng để đảm bảo rằng yêu cầu gửi lên server thực sự là do người dùng thực hiện, nhưng lỗ hổng xảy ra khi mã này *bị thiếu trong Form hoặc không được gửi kèm trong phần Header đối với các ứng dụng xác thực bằng cơ chế cookie-session*. Do đó, khóa luận này là quá trình nghiên cứu các phương pháp vừa mở rộng phạm vi kiểm thử trên ứng dụng mục tiêu, vừa mở rộng công cụ để giúp nâng cấp khả năng phát hiện lỗ hổng, giúp đáp ứng tốt hơn yêu cầu bảo mật cho các ứng dụng Web.

1.4 Mục tiêu và đóng góp của khóa luận

Khóa luận hướng tới mục tiêu mở rộng khả năng phát hiện lỗ hổng đối với các loại lỗ hổng mà phiên bản hiện tại của công cụ DAST được lựa chọn là OWASP ZAP còn hạn chế. Cùng với đó, khóa luận tập trung tối ưu quy trình kiểm thử bảo mật trên ứng dụng Web SPA, khai thác tối đa khả năng của công cụ và tận dụng các tiến bộ của trí tuệ nhân tạo để nghiên cứu và triển khai một quy trình kiểm thử bảo mật được vận hành tự động với tác tử AI. Từ những mục tiêu này, khóa luận đặt ra các câu hỏi nghiên cứu (Research Question - RQ) sau:

- **RQ1:** Những hạn chế còn tồn tại trong phiên bản mặc định của công cụ OWASP ZAP là gì, hướng tiếp cận như thế nào để khắc phục các hạn chế này, phương pháp cụ thể của từng hướng tiếp cận được triển khai như thế nào?
- **RQ2:** Công cụ kiểm thử OWASP ZAP trong phiên bản mặc định đạt được hiệu quả phát hiện lỗ hổng bảo mật Web như thế nào, các phương pháp cải tiến được đề xuất có thể giúp nâng cao độ hiệu quả của công cụ tới mức nào?
- **RQ3:** Việc ứng dụng trí tuệ nhân tạo, tác tử AI với quy trình kiểm thử bảo mật động mang lại những lợi ích và thách thức phát sinh gì? Mức độ phù hợp của các phương pháp này trong các điều kiện vận hành thực tế được đánh giá ra sao?

Dựa trên các câu hỏi nghiên cứu, khóa luận đề xuất các giải pháp tương ứng và có được những đóng góp chính sau:

- Khóa luận phân tích các hạn chế của OWASP ZAP trong cấu hình mặc định, trình bày hướng tiếp cận và triển khai phương pháp nhằm khắc phục các hạn chế này. Đóng góp này với mục đích đưa ra câu trả lời cho RQ1, cụ thể bao gồm: bổ sung các luật cho những lỗ hổng thuộc nhóm TOP 25 CWE mà công cụ chưa hỗ trợ hoặc có hỗ trợ nhưng chưa hiệu quả, triển khai pipeline kiểm thử kết hợp công cụ tương tác UI tự động giúp công cụ ZAP mở rộng site-map và tăng phạm vi kiểm thử.
- Đưa ra đánh giá về khả năng phát hiện lỗ hổng của ZAP trong trạng thái mặc định và phân tích mức độ cải thiện khi áp dụng các phương pháp được

đề xuất. Đóng góp này nhằm đưa ra câu trả lời cho RQ2, được thể hiện thông qua quá trình thực nghiệm trên nhiều kịch bản ứng dụng Web, làm rõ sự thay đổi về hiệu quả phát hiện lỗ hổng trên từng ứng dụng Web mục tiêu ở hai giai đoạn trước và sau cải tiến.

- Đánh giá về hiệu quả của các công cụ LLM và tác tử AI khi kết hợp chúng vào quy trình kiểm thử bảo mật động. Đồng thời phân tích các thách thức, mức độ khả thi khi vận hành thực tế. Mục đích của đóng góp này là đưa ra câu trả lời cho RQ3.

Với những câu hỏi nghiên cứu và đóng góp nêu trên, phần còn lại của khóa luận được tổ chức và trình bày với nội dung:

- **Chương 2:** Cung cấp những kiến thức cơ bản nhất về bảo mật cho ứng dụng Web, các loại lỗ hổng bảo mật phổ biến, kiến thức về mô hình ngôn ngữ, tổng quan về các phương pháp kiểm thử bảo mật và giới thiệu về các công cụ DAST cũng như đưa ra lý do lựa chọn công cụ OWASP ZAP.
- **Chương 3:** Chương này trình bày các hướng tiếp cận chủ đạo và mô hình hóa những giải pháp nhằm nâng cao hiệu quả của công cụ OWASP ZAP, với trọng tâm là cải thiện khả năng phát hiện lỗ hổng. Mỗi giải pháp được mô tả một cách hệ thống, bao gồm ý tưởng nền tảng, luồng hoạt động và quy trình triển khai tương ứng.
- **Chương 4:** Trình bày cấu hình thực nghiệm, các độ đo, phương pháp đánh giá và kết quả của các thực nghiệm tương ứng để chứng minh hiệu quả của các giải pháp đề xuất.
- **Kết luận:** Tóm tắt những kết quả chính, đánh giá những điểm mạnh, điểm yếu của giải pháp, trả lời các câu hỏi nghiên cứu và đề xuất các hướng đi mới trong tương lai để mở rộng và phát triển giải pháp.

Chương 2

Kiến thức cơ sở

Chương này cung cấp những kiến thức nền tảng về bảo mật cho ứng dụng Web, các loại lỗ hổng bảo mật phổ biến hiện nay và tổng quan về các phương pháp kiểm thử bảo mật. Nội dung sẽ tập trung giới thiệu các công cụ kiểm thử bảo mật động, đồng thời đưa ra lý do lựa chọn công cụ OWASP ZAP để cải tiến. Bên cạnh đó, chương này cũng trình bày tổng quan các kiến thức nền tảng về mô hình ngôn ngữ lớn và công cụ Langflow được sử dụng để xây dựng ứng dụng hỏi đáp hỗ trợ, sinh thêm luật kiểm thử cho công cụ ZAP và đưa ra hướng dẫn kiểm thử các loại lỗ hổng phổ biến trên ứng dụng Web. Cuối cùng, chương này giới thiệu công cụ Browser Use — một giải pháp tự động hóa UI dựa trên tác tử AI.

2.1 Kiểm thử bảo mật cho ứng dụng web

2.1.1 Top 10 OWASP và Top 25 CWE cho ứng dụng Web

Các lỗ hổng bảo mật trên ứng dụng Web rất đa dạng và không ngừng phát triển, có những loại hình tấn công đã trở nên phổ biến và được cộng đồng an ninh mạng, đặc biệt là tổ chức chuyên nghiên cứu về lỗ hổng bảo mật cho ứng dụng Web OWASP (Open Web Application Security Project) cảnh báo thường xuyên. Các lỗ hổng này thường được tổng hợp và cập nhật trong danh sách Top 10 OWASP, một tài liệu chuẩn giúp các nhà phát triển và chuyên gia bảo mật nhận diện những rủi ro nghiêm trọng nhất mà ứng dụng Web có thể gặp phải. Việc nhận biết và hiểu rõ cơ chế hoạt động của từng loại lỗ hổng trong danh sách này là cơ sở để nghiên cứu và đưa ra giải pháp cho bài toán cải thiện độ hiệu quả của công cụ DAST.

Dưới đây là 10 lỗ hổng trong danh sách OWASP Top 10 (2021) [13] với mô tả khái quát về lỗ hổng, một vài đối chiếu với các lỗ hổng trong OWASP Top 10 (2017) và ví dụ về hậu quả mà nó gây ra tới hệ thống:

- **A01:2021 - Broken Access Control:** Lỗi hổng này xuất hiện khi các quy tắc kiểm soát truy cập không được triển khai đúng cách, cho phép kẻ tấn công thực hiện các hành động trái phép ngoài phạm vi quyền hạn của họ. Ví dụ, một người dùng có thể thay đổi URL để truy cập vào hồ sơ của người dùng khác hoặc trang quản trị. Hậu quả có thể là lộ dữ liệu nhạy cảm hoặc chiếm quyền điều khiển tài khoản.
- **A02:2021 - Cryptographic Failures:** Lỗi hổng này, trước đây là Sensitive Data Exposure, xảy ra khi các dữ liệu nhạy cảm không được mã hóa hoặc mã hóa yếu. Ví dụ, việc lưu trữ mật khẩu dưới dạng văn bản thuần hoặc sử dụng các giao thức truyền tải không an toàn như HTTP, cho phép kẻ tấn công dễ dàng đọc và đánh cắp dữ liệu. Một ví dụ về hậu quả nghiêm trọng là vụ tấn công vào Equifax, làm lộ thông tin cá nhân của hàng triệu người dùng [14].
- **A03:2021 - Injection:** Lỗi hổng này là một trong những loại lỗi hổng phổ biến và lâu đời nhất. Nó xảy ra khi dữ liệu không đáng tin cậy được gửi đến một trình thông dịch như là một phần của lệnh hoặc truy vấn, cho phép kẻ tấn công thực thi mã độc. Ví dụ điển hình là SQL Injection, cho phép kẻ tấn công thực thi các truy vấn SQL trái phép để lấy hoặc xóa toàn bộ cơ sở dữ liệu khách hàng.
- **A04:2021 - Insecure Design:** Một danh mục mới trong phiên bản 2021, tập trung vào các lỗi hổng liên quan đến những khiếm khuyết trong thiết kế kiến trúc hoặc mô hình bảo mật. Đây không phải là lỗi lập trình mà là những rủi ro bảo mật vốn có trong thiết kế hệ thống, khó có thể vá lỗi bằng cách sửa mã nguồn.
- **A05:2021 - Security Misconfiguration:** Nhóm lỗi này rất phổ biến và dễ bị khai thác, xuất phát từ việc cấu hình sai máy chủ, framework, hệ điều hành hoặc ứng dụng. Hậu quả có thể là lộ các thư mục nhạy cảm, sử dụng tài khoản mặc định không an toàn, hoặc không vá các bản cập nhật bảo mật kịp thời.
- **A06:2021 - Vulnerable and Outdated Components:** Trước đây là Using Components with Known Vulnerabilities, lỗi hổng này xảy ra khi các nhà phát triển sử dụng các thư viện, framework hoặc các thành phần mã nguồn mở có chứa lỗi hổng bảo mật đã được công bố mà không được cập nhật kịp thời. Vụ

tấn công vào Apache Struts là một ví dụ điển hình [15], khi lỗ hổng trong một thư viện đã cho phép kẻ tấn công chiếm quyền điều khiển máy chủ.

- **A07:2021 - Identification and Authentication Failures:** Lỗ hổng này liên quan đến việc triển khai xác thực hoặc quản lý phiên không đúng cách, cho phép kẻ tấn công mạo danh người dùng hoặc truy cập trái phép. Ví dụ, việc sử dụng các thông tin đăng nhập mặc định hoặc quản lý phiên yếu kém có thể dẫn đến việc chiếm đoạt tài khoản người dùng và thực hiện các giao dịch gian lận.
- **A08:2021 - Software and Data Integrity Failures:** Một danh mục mới trong OWASP Top 10 (2021) tập trung vào các lỗ hổng liên quan đến việc thiếu kiểm tra tính toàn vẹn của phần mềm và dữ liệu. Ví dụ, việc tự động cập nhật phần mềm không được kiểm tra chữ ký số có thể cho phép kẻ tấn công chèn các mã độc vào bản cập nhật.
- **A09:2021 - Security Logging and Monitoring Failures:** Lỗ hổng này xuất hiện khi ứng dụng không ghi lại các sự kiện quan trọng hoặc không có hệ thống giám sát hiệu quả. Điều này làm cho việc phát hiện và ứng phó với các cuộc tấn công trở nên khó khăn hơn nhiều, cho phép kẻ tấn công hoạt động trong một thời gian dài mà không bị phát hiện.
- **A10:2021 - Server-Side Request Forgery (SSRF):** Lỗ hổng SSRF là một loại lỗ hổng mới được thêm vào danh sách, nơi kẻ tấn công có thể khiến máy chủ web thực hiện các yêu cầu đến các tài nguyên nội bộ hoặc bên ngoài không mong muốn. Điều này có thể dẫn đến việc truy cập trái phép các tài nguyên bị hạn chế hoặc thực hiện các cuộc tấn công từ bên trong mạng lưới.

Bên cạnh OWASP Top 10, khóa luận còn tham khảo danh sách **CWE Top 25 Most Dangerous Software Weaknesses** [16], do MITRE tổng hợp, nhằm nhận diện các lỗ hổng phần mềm phổ biến và nguy hiểm nhất trên nhiều loại ứng dụng, không chỉ riêng Web. Trong khi OWASP Top 10 tập trung vào các lỗ hổng thường gặp trong ứng dụng Web và cung cấp hướng dẫn phòng ngừa ở mức độ tổng quan. Top 25 CWE đi sâu hơn vào các loại lỗ hổng, mô tả chi tiết cơ chế, nguyên nhân, và các biến thể khai thác.

Danh sách Top 25 CWE nói riêng và CWE nói chung là cơ sở cho các công cụ kiểm thử triển khai cơ chế phát hiện lỗ hổng phần mềm, đồng thời được hầu hết các công cụ DAST tham chiếu trong báo cáo. Ví dụ, các lỗ hổng phổ biến trong tài liệu CWE bao gồm:

- **CWE-90 Improper Neutralization of Special Elements used in an LDAP Query ('LDAP Injection')** [17]: Lỗ hổng này xảy ra khi ứng dụng Web chèn trực tiếp dữ liệu đầu vào của người dùng vào truy vấn LDAP mà không kiểm tra hoặc mã hóa ký tự đặc biệt một cách thích hợp. Điều này cho phép kẻ tấn công thay đổi cấu trúc truy vấn LDAP gốc, từ đó truy xuất, sửa đổi hoặc tiết lộ các thông tin nhạy cảm được lưu trữ trong dịch vụ thư mục như OpenLDAP hoặc Active Directory. Hậu quả có thể bao gồm rò rỉ thông tin người dùng, quyền truy cập, cấu hình hệ thống, trong trường hợp nghiêm trọng thì dẫn tới chiếm quyền điều khiển ứng dụng.
- **CWE-200 Exposure of Sensitive Information to an Unauthorized Actor** [18]: Lỗ hổng này mô tả tình trạng ứng dụng vô tình tiết lộ thông tin nhạy cảm cho người dùng hoặc các đối tượng không được phép truy cập. Dữ liệu bị lộ có thể bao gồm thông tin cá nhân, cấu hình hệ thống, khóa truy cập, thông tin máy chủ hoặc chi tiết lỗi nội bộ. Mặc dù các công cụ quét bảo mật như OWASP ZAP thường tích hợp quy tắc phát hiện rò rỉ thông tin cơ bản, nhưng trong thực tế, nhiều tình huống rò rỉ mang tính đặc thù theo nghiệp vụ hoặc cấu trúc phản hồi của ứng dụng, khiến các quy tắc mặc định khó nhận diện đầy đủ.
- **CWE-352 Cross-Site Request Forgery (CSRF)** [19]: Lỗ hổng CWE-352 xảy ra khi kẻ tấn công lợi dụng phiên đăng nhập hợp lệ của người dùng để gửi các yêu cầu độc hại đến máy chủ mà không có sự đồng ý của nạn nhân. Trình duyệt sẽ tự động đính kèm cookie hoặc mã xác thực phiên vào yêu cầu giả mạo, khiến hệ thống xử lý như một hành động hợp lệ. Hậu quả có thể dẫn đến thay đổi dữ liệu, chỉnh sửa thông tin người dùng hoặc thực hiện các thao tác quản trị ngoài ý muốn. Kiểu tấn công này thường nhắm vào các hành động thay đổi trạng thái ứng dụng, tương ứng với các phương thức HTTP như POST, PUT, DELETE hoặc PATCH. Trong các hệ thống sử dụng cơ chế xác thực dựa trên cookie-session, nếu không triển khai biện pháp bảo vệ như

CSRF Token hoặc cấu hình Cookie SameSite, kẻ tấn công có thể dễ dàng giả mạo yêu cầu và thực thi các hành động trái phép dưới danh tính người dùng.

- **CWE-93 Improper Neutralization of CRLF Sequences ('CRLF Injection')** [20]: Lỗi hỏng CWE-93 là dạng tấn công cho phép kẻ tấn công chèn ký tự điều khiển dòng mới vào dữ liệu đầu vào của người dùng. Khi ứng dụng Web không xử lý đúng các ký tự Carriage Return (`\r`) hoặc Line Feed (`\n`), máy chủ có thể thực thi dữ liệu chứa các ký tự này như một phần của tiêu đề HTTP. Khi đó, kẻ tấn công có thể phá vỡ cấu trúc phản hồi, kết thúc tiêu đề hiện tại và chèn thêm tiêu đề hoặc nội dung mới. Hậu quả nghiêm trọng là phản hồi của máy chủ bị tách thành hai phần riêng biệt, tạo điều kiện thực hiện các hành vi độc hại như chèn tiêu đề Set-Cookie giả mạo, tấn công XSS bằng cách chèn mã JavaScript vào phần thân phản hồi hoặc thực hiện Cache Poisoning tại các proxy trung gian.

Như vậy, cả Top 10 OWASP và Top 25 CWE đều nhằm mục tiêu nâng cao nhận thức về các lỗi hỏng phổ biến, hướng dẫn phòng ngừa và cải thiện chất lượng bảo mật của phần mềm. Điểm khác biệt cơ bản là Top 10 OWASP thiên về khía cạnh ứng dụng Web và mức độ rủi ro tổng quan, trong khi CWE Top 25 mang tính kỹ thuật, chi tiết và bao quát nhiều loại ứng dụng khác nhau, từ Web đến Desktop hay các dịch vụ Backend. Việc lựa chọn Top 10 OWASP và Top 25 CWE làm cơ sở để đề xuất và đánh giá giải pháp, giúp khóa luận thu hẹp phạm vi để tập trung vào những lỗi hỏng phổ biến trên ứng dụng Web.

2.1.2 Các phương pháp để kiểm thử bảo mật trên ứng dụng web

Để đảm bảo chất lượng bảo mật cho ứng dụng Web, nhiều phương pháp kiểm thử bảo mật đã và đang được nghiên cứu, phát triển. SAST là kiểm thử “hộp trắng”, phân tích mã nguồn mà không cần thực thi ứng dụng, hiệu quả trong việc phát hiện lỗi sớm. IAST là kiểm thử “hộp xám”, kết hợp phân tích mã nguồn và quan sát luồng dữ liệu trong môi trường ứng dụng được vận hành, giúp nhận diện lỗ hổng chính xác hơn. Bên cạnh đó còn phương pháp kiểm thử thâm nhập thủ công do các chuyên gia thực hiện đóng vai trò quan trọng trong việc phát hiện các lỗ hổng logic hoặc phức tạp mà công cụ tự động khó nhận diện. Mỗi phương pháp đều có ưu, nhược điểm và phù hợp ở từng giai đoạn khác nhau trong quy trình phát triển phần mềm. Trong phạm vi khóa luận, trọng tâm của nghiên cứu sẽ chỉ tập trung vào DAST.

Kiểm thử bảo mật động (DAST) là phương pháp “hộp đen” thực thi ứng dụng và tấn công nó từ bên ngoài, mô phỏng hành vi của một kẻ tấn công thực sự. DAST được thực hiện trên một ứng dụng đang hoạt động, có thể là trong môi trường thử nghiệm hoặc thậm chí là môi trường sản phẩm. Quá trình kiểm thử bảo mật động thường bao gồm các bước chính:

- **Giai đoạn khám phá:** DAST sẽ khai thác ứng dụng Web để tìm kiếm tất cả các trang, đường dẫn, tham số và các điểm vào khác. Mục tiêu của giai đoạn này là xây dựng sơ đồ URL Web đầy đủ phục vụ cho quá trình quét bảo mật ở các bước tiếp theo.
- **Giai đoạn quét bảo mật:** Ở giai đoạn này, DAST tiến hành hai cơ chế kiểm thử là: quét thụ động và quét chủ động.
 - **Quét thụ động:** DAST chỉ quan sát và phân tích các thông tin trong luồng HTTP/HTTPS mà không can thiệp vào tham số hoặc gửi payload tấn công. Mục tiêu là phát hiện các dấu hiệu rủi ro bảo mật dựa trên các thông tin từ phản hồi và hành vi của ứng dụng.
 - **Quét chủ động:** DAST chủ động tạo và gửi các payload tấn công vào các điểm vào đã được xác định ở giai đoạn khám phá. Thông qua phân tích phản hồi của ứng dụng và áp dụng các luật nhận diện đặc thù của từng loại lỗ hổng, DAST đưa ra cảnh báo về các điểm yếu bảo mật tiềm ẩn.

- **Giai đoạn báo cáo:** Giai đoạn cuối cùng này sẽ tổng hợp tất cả các cảnh báo, cung cấp thông tin chi tiết về loại lỗ hổng, vị trí tìm thấy, và mức độ nghiêm trọng.

Có hai phương pháp kiểm thử bảo mật động phổ biến [21] hiện nay mà các công cụ đang sử dụng để triển khai:

- **Signature-based Detection:** Đây là phương pháp truyền thống, tương tự như cách hoạt động của các phần mềm diệt virus. Công cụ sử dụng một cơ sở dữ liệu chứa các mẫu của các cuộc tấn công đã biết và các phản hồi lỗi tương ứng. Khi kiểm thử, công cụ sẽ gửi các payload tấn công đã được định nghĩa trước và so sánh phản hồi của ứng dụng với các mẫu đã có trong cơ sở dữ liệu. Ví dụ, một công cụ như Nuclei hoạt động theo phương pháp này. Nuclei sử dụng các mẫu tấn công được viết bằng YAML-based Domain Specific Language để gửi các yêu cầu HTTP được thiết kế đặc biệt và tìm kiếm các dấu hiệu cụ thể trong phản hồi. Ví dụ, một chuỗi văn bản đặc trưng hoặc một thay đổi mã trạng thái để xác định lỗ hổng. Ưu điểm của phương pháp này là tốc độ nhanh và dễ dàng mở rộng bằng cách thêm các mẫu mới. Tuy nhiên, nó không thể phát hiện các lỗ hổng chưa được công bố (zero-day) hoặc các lỗ hổng logic phức tạp.
- **Anomaly-based Detection:** Đây là một phương pháp hiện đại hơn, không chỉ dựa vào các mẫu tấn công đã biết mà còn phân tích hành vi của ứng dụng để tìm ra các bất thường. Thay vì chỉ tìm kiếm một mẫu tấn công cụ thể đã có, công cụ sẽ phân tích cách ứng dụng phản ứng với các payload khác nhau. Ví dụ, khi một payload tấn công SQL Injection được gửi, công cụ sẽ phân tích thời gian phản hồi của cơ sở dữ liệu, sự thay đổi trong nội dung trang, hoặc các lỗi ngoại lệ được trả về để xác định liệu có lỗ hổng tồn tại hay không. Phương pháp này thường sử dụng các kỹ thuật như “Blind SQL injection” hoặc “Time-based blind SQL injection” để xác định lỗ hổng. ZAP là một công cụ sử dụng cả hai phương pháp này, nhưng phần lớn các tính năng quét chủ động của nó đều dựa trên việc phân tích hành vi phản hồi của ứng dụng. Ưu điểm của phương pháp này là khả năng phát hiện các lỗ hổng chưa được biết đến (zero-day) và các lỗ hổng logic, nhưng có thể chậm hơn và tạo ra nhiều cảnh báo giả hơn nếu không được cấu hình đúng cách.

Như vậy, DAST giữ vai trò quan trọng trong việc đánh giá an toàn ứng dụng từ góc nhìn của kẻ tấn công. Tuy nhiên, phương pháp này vẫn còn những hạn chế, đặc biệt là về mức độ bao phủ các lỗ hổng trong OWASP Top 10 và CWE Top 25.

2.2 Công cụ kiểm thử bảo mật động

Mục này giới thiệu và phân tích các công cụ kiểm thử bảo mật động cho ứng dụng Web, đồng thời làm rõ cơ sở lựa chọn OWASP ZAP. Để xác định công cụ nền tảng phù hợp cho nghiên cứu và phát triển cải tiến, khóa luận đã khảo sát bốn công cụ là Burp Suite, Wapiti, Nuclei và OWASP ZAP.

Burp Suite [12] là công cụ thương mại mạnh, được sử dụng rộng rãi trong kiểm thử bảo mật. Tuy nhiên, do mã nguồn đóng, công cụ không cho phép can thiệp hay tùy chỉnh ở mức lõi để phục vụ mục tiêu nghiên cứu. Bên cạnh đó, phiên bản đầy đủ tính năng (Burp Suite Professional/Enterprise) yêu cầu giấy phép thương mại và giới hạn số lượng người dùng, dẫn đến việc không thể triển khai đánh giá trên quy mô phù hợp với phạm vi nghiên cứu. Vì những hạn chế này, Burp Suite không được lựa chọn làm nền tảng phát triển trong khóa luận.

Wapiti [22] là công cụ DAST mã nguồn mở, hoạt động tương tự ZAP, nhưng hạn chế về giao diện, cộng đồng nhỏ và tài liệu ít, khiến việc tùy chỉnh khó khăn. Kiến trúc của Wapiti tập trung vào engine chính, không hỗ trợ mở rộng độc lập, việc bổ sung kỹ thuật kiểm thử mới thường yêu cầu chỉnh sửa trực tiếp mã nguồn lõi, làm tăng độ phức tạp và khó bảo trì. Do đó, đặt trong bối cảnh so sánh với ZAP, Wapiti không được chọn để triển khai nâng cấp nhằm đạt mục tiêu nghiên cứu của khóa luận.

Nuclei [23] cũng là một công cụ mã nguồn mở mới, nhẹ và được tối ưu cho việc quét nhanh trên quy mô lớn (ví dụ: trong môi trường CI/CD, DevSecOps pipeline) nhằm phục vụ cho mục đích quét nhanh và tối ưu tài nguyên tự động hóa. Tuy nhiên, nó thiếu các tính năng khám phá toàn diện mà công cụ DAST cần có, thường phải dựa vào đầu ra URL của các công cụ khác (như ZAP hoặc Wapiti) để thực hiện kiểm thử. Vì mục tiêu chính của khóa luận là nâng cao khả năng phát hiện các lỗ hổng bảo mật nên Nuclei không phải là lựa chọn ưu tiên.

OWASP ZAP [11] được lựa chọn làm công cụ nghiên cứu chính nhờ sở hữu engine quét mạnh, khả năng khám phá và phát hiện lỗ hổng hiệu quả hơn so với các công

cụ đã khảo sát, cùng ưu thế mã nguồn mở cho phép phân tích và tùy chỉnh sâu. ZAP hoạt động với cơ chế proxy, hỗ trợ chặn, phân tích và điều chỉnh thông tin HTTP/HTTPS, cung cấp đầy đủ cơ chế quét tự động, khám phá cấu trúc ứng dụng, tấn công chủ động và phân tích thụ động. Khả năng mở rộng vượt trội thông qua hệ thống plugin, scripting và API, kết hợp với cộng đồng lớn và cập nhật thường xuyên.

Sự kết hợp giữa các yếu tố: mã nguồn mở, cộng đồng lớn và khả năng mở rộng đã khiến OWASP ZAP được lựa chọn để triển khai các thử nghiệm và phát triển các giải pháp cải tiến trong phạm vi khóa luận này. Mặc dù ZAP mạnh mẽ, nó vẫn đối mặt với những hạn chế chung của DAST, đặc biệt là về khả năng bao phủ đối với các ứng dụng Web SPA, cũng như hạn chế về khả năng phát hiện các lỗ hổng bảo mật phổ biến. Việc mở rộng khả năng phát hiện lỗ hổng của ZAP và kiểm thử hiệu quả trên các loại ứng dụng Web SPA là trọng tâm nghiên cứu.

2.3 Mô hình ngôn ngữ lớn

2.3.1 Cơ chế hoạt động của mô hình ngôn ngữ lớn

Mô hình ngôn ngữ lớn (LLM) là một dạng mô hình học máy được huấn luyện trên tập dữ liệu văn bản khổng lồ, có khả năng xử lý, hiểu và sinh ngôn ngữ tự nhiên. LLM hoạt động dựa trên kiến trúc Transformer, với cơ chế self-attention được sử dụng để xác định mối quan hệ giữa các từ trong câu, cho phép mô hình nắm bắt ngữ cảnh và tạo phản hồi. Khi tiếp nhận một câu hỏi hoặc lệnh, LLM sẽ phân tích ngữ cảnh, dự đoán từ tiếp theo dựa trên xác suất và lặp lại quá trình này cho đến khi hình thành phản hồi hoàn chỉnh [24].

Trong quá trình suy luận, hành vi và chất lượng phản hồi của LLM được điều chỉnh thông qua các cơ chế kiểm soát đầu vào. *System Message* được chèn ở đầu chuỗi ngữ cảnh và có vai trò thiết lập các ràng buộc hành vi, phạm vi hoạt động và phong cách phản hồi của mô hình trong suốt phiên làm việc; thành phần này có mức độ ưu tiên cao và luôn được mô hình tuân thủ ngay cả khi mâu thuẫn với các đầu vào khác. *Prompt* là các yêu cầu tác vụ do người dùng cung cấp ở từng lượt tương tác, chỉ ảnh hưởng đến phản hồi của mô hình trong ngữ cảnh nhiệm vụ đó mà không thay đổi hành vi tổng thể đã được thiết lập bởi *System Message* [25]. Tham số *Temperature* điều tiết đặc tính của phân phối xác suất khi chọn token

tiếp theo: giá trị thấp tạo phản hồi mang tính xác định và ổn định hơn, còn giá trị cao tăng mức đa dạng nhưng có thể làm giảm tính nhất quán [26].

Với khả năng hiểu ngữ cảnh và sinh phản hồi phù hợp, LLM mở ra nhiều ứng dụng tiềm năng cho nhiều lĩnh vực, trong đó có an ninh mạng và kiểm thử bảo mật [27]. Trong bối cảnh khóa luận, LLM có thể góp phần nâng cao mức độ tự động hóa và giúp tiết kiệm thời gian để nâng cấp khả năng phát hiện lỗ hổng bảo mật của công cụ kiểm thử bảo mật động OWASP ZAP. Cụ thể, LLM được ứng dụng để:

- Phân tích và hiểu ngữ cảnh ngôn ngữ tự nhiên, từ đó đưa ra quyết định thực hiện các tương tác tự động khám phá Web, đảm bảo phù hợp với logic nghiệp vụ, quá trình này giúp bổ sung thêm URL cho site-map của ZAP.
- Sinh luật kiểm thử cho ZAP, hỗ trợ người kiểm thử lần đầu tiếp cận các lỗ hổng bảo mật, hướng dẫn khai thác công cụ, đồng thời tạo và hướng dẫn tích hợp các đoạn mã bổ sung phục vụ quá trình kiểm thử.

2.3.2 Các công cụ làm việc với mô hình ngôn ngữ lớn

Để triển khai các ứng dụng LLM trong nghiên cứu này, một số công cụ và kỹ thuật điều chỉnh LLM đã được áp dụng. Trong đó, **Langflow** [28] là một nền tảng mã nguồn mở mạnh mẽ, hỗ trợ xây dựng và triển khai ứng dụng dựa trên LLM thông qua giao diện kéo – thả trực quan. Nền tảng này giúp đơn giản hóa quá trình phát triển và thử nghiệm các kỹ thuật xử lý LLM, bao gồm cả việc triển khai mô hình LLM RAG [29]. Sau khi cấu hình và cung cấp luồng dữ liệu cho ứng dụng RAG với mẫu các thành phần kéo - thả có sẵn, người dùng có thể trực tiếp thực hiện hỏi – đáp trên **Langflow**, nhờ đó việc thử nghiệm và triển khai các kỹ thuật nâng cao trên LLM trở nên thuận tiện hơn với công cụ low-code này.

Ngoài **Langflow**, nghiên cứu còn sử dụng **Browser Use** [30], một công cụ tự động hóa trình duyệt sử dụng mô tả ngôn ngữ tự nhiên. Công cụ này cho phép đầu vào là các kịch bản thao tác trên ứng dụng Web cần khai thác, chẳng hạn như luồng hoạt động hoặc dữ liệu thông tin đăng nhập để truy cập các chức năng bên trong. Dựa vào các thông tin trên, LLM đưa ra quyết định cho **Browser Use** tiến hành tự động tương tác trên trình duyệt và tự động khám phá ứng dụng Web. Từ đó, công cụ hỗ trợ hiệu quả cho bài toán DAST đối với các ứng dụng Web SPA có logic nghiệp vụ phức tạp và đòi hỏi nhiều tương tác thực tế.

Chương 3

Phương pháp cải thiện độ hiệu quả trong kiểm thử bảo mật trên ứng dụng web

Chương số 3 của khóa luận sẽ trình bày chi tiết các phương pháp được đề xuất nhằm tăng cường hiệu quả của công cụ kiểm thử bảo mật động cho ứng dụng Web. Nội dung chính của chương được xây dựng để trực tiếp trả lời câu hỏi nghiên cứu RQ1, tập trung vào việc xác định các hạn chế của công cụ ZAP hiện tại, từ đó nêu ra hướng tiếp cận và phương pháp triển khai để khắc phục các hạn chế đó.

3.1 Hướng tiếp cận

Dựa trên nguyên lý hoạt động của công cụ ZAP được giới thiệu ở chương trước và những hạn chế của DAST, chương này sẽ đề xuất hai hướng tiếp cận chính. Các hướng tiếp cận này nhằm mục đích cải thiện phạm vi phát hiện lỗ hổng trong ứng dụng Web mục tiêu và nâng cao khả năng phát hiện các lỗ hổng phổ biến, đặc biệt là những lỗ hổng thuộc Top 25 CWE và Top 10 OWASP.

Phương pháp thứ nhất xuất phát từ việc nhận diện một hạn chế trong hiệu quả hoạt động của công cụ ZAP khi kiểm thử Web SPA, đó là khả năng xây dựng site-map còn nhiều giới hạn. Nguyên nhân chủ yếu nằm ở hiệu suất chưa cao của các thành phần Spider và Ajax Spider trong việc xử lý nội dung sinh động được tải và hiển thị bởi trình duyệt. Do đặc trưng của loại ứng dụng này phụ thuộc mạnh vào cơ chế định tuyến phía client và quá trình tải nội dung động, các thuật toán khám phá mặc định của công cụ ZAP chưa đáp ứng tốt yêu cầu thu thập URL trên ứng dụng Web SPA mục tiêu. Hệ quả là công cụ không xây dựng được đầy đủ site-map, làm giảm phạm vi kiểm thử và tăng nguy cơ bỏ sót lỗ hổng.

Nhằm khắc phục điểm yếu này, khóa luận đề xuất hướng tiếp cận thứ nhất, tập trung nâng cao khả năng khám phá ứng dụng cho công cụ kiểm thử. Ý tưởng là tận dụng ZAP làm máy chủ trung gian, ghi nhận và phân tích toàn bộ lưu lượng trao đổi giữa trình duyệt và máy chủ Web. Thực nghiệm kết hợp ZAP với thao tác

thủ công và hướng tới triển khai phương pháp tự động tương tác trên trình duyệt theo kịch bản để mô phỏng hành vi người dùng. Mọi yêu cầu và phản hồi sinh ra trong quá trình mô phỏng đều được lưu lại, từ đó xây dựng một site-map chứa đầy đủ thông tin và các đường dẫn. Bản đồ này cung cấp cho bộ quét ZAP danh sách URL toàn diện, giúp mở rộng không gian tấn công, tăng độ bao phủ kiểm thử và phát hiện nhiều cảnh báo lỗ hổng hơn.

Bên cạnh hạn chế về khả năng khám phá ứng dụng đã được đề cập trong hướng tiếp cận thứ nhất, một giới hạn khác của kiểm thử bảo mật động nằm ở khả năng nhận diện lỗ hổng khi dựa vào tập luật quét mặc định của công cụ OWASP ZAP. Mặc dù ZAP đã được trang bị sẵn nhiều quy tắc kiểm thử nhằm phát hiện các lỗ hổng phổ biến, nhưng những quy tắc này vẫn còn nhiều thiếu sót, chưa đủ khả năng nhận diện các dạng lỗ hổng mới và những biến thể phức tạp của cùng một lỗ hổng. Điều này khiến một số lỗ hổng quan trọng trong danh sách TOP 25 CWE, TOP 10 OWASP không được phát hiện trong quá trình quét, dẫn đến việc công cụ bỏ sót các cảnh báo tương ứng.

Nhằm giải quyết hạn chế này, hướng tiếp cận thứ hai của khóa luận tập trung vào việc mở rộng và tinh chỉnh các luật phát hiện lỗ hổng, hướng tiếp cận này hướng tới mục tiêu nâng cao khả năng phát hiện lỗ hổng của công cụ so với cấu hình mặc định. Cụ thể, khóa luận tiến hành xây dựng thêm các tập luật tùy chỉnh dưới dạng các đoạn mã nguồn bổ sung cho công cụ ZAP. Các tập lệnh này được phát triển bằng ngôn ngữ JavaScript và thực thi trên công cụ ZAP với GraalVM [31], cho phép tương tác trực tiếp với lõi của công cụ, từ đó tận dụng các API sẵn có trong ZAP. Tuy nhiên, việc phát triển các luật kiểm thử bằng cách thủ công đòi hỏi kiến thức chuyên sâu về bảo mật, hiểu rõ cấu trúc nội bộ của ZAP và tốn nhiều thời gian để phân tích, viết, và kiểm thử cho từng đoạn mã. Để khắc phục vấn đề này, khóa luận ứng dụng mô hình ngôn ngữ lớn kết hợp với kỹ thuật tạo tăng cường truy xuất (RAG) nhằm hỗ trợ cho quá trình sinh luật. Trong giai đoạn thực nghiệm, mô hình ngôn ngữ được cung cấp ngữ cảnh truy xuất từ cơ sở dữ liệu tri thức gồm OWASP WSTG [32], các tập luật ZAP mẫu từ cộng đồng [33], từ đó sinh ra các đoạn mã kiểm thử phù hợp với từng loại lỗ hổng. Phương pháp hỗ trợ sinh luật kiểm thử này tạo điều kiện để rút ngắn đáng kể thời gian cần thiết để tiếp cận và phát triển một quy tắc quét bảo mật mới.

Hướng tiếp cận thứ nhất giúp công cụ mở rộng phạm vi khảo sát ứng dụng, còn hướng tiếp cận thứ hai nâng cao khả năng nhận diện các lỗ hổng đặc thù. Nhờ đó, quá trình kiểm thử trở nên hiệu quả hơn, cải thiện độ bao phủ các lỗ hổng bảo mật của công cụ và đáp ứng tốt hơn yêu cầu kiểm thử an toàn thông tin đối với các ứng dụng Web hiện nay.

3.2 Cách thức thực hiện

Các nội dung trong mục này sẽ trình bày cụ thể cách thức triển khai chính của phương pháp, bao gồm việc sử dụng các kỹ thuật để xây dựng site-map đầy đủ trước khi tiến hành quét bảo mật, triển khai thành luồng kiểm thử bảo mật tự động, phối hợp nhiều công cụ làm tăng độ hiệu quả khi kiểm thử trên ứng dụng Web SPA. Cuối cùng, chương này tập trung vào bổ sung các luật quét bảo mật chưa có trên phiên bản mặc định của công cụ, giúp công cụ cải thiện khả năng bao phủ các lỗ hổng bảo mật phổ biến nằm trong Top 25 CWE và Top 10 OWASP.

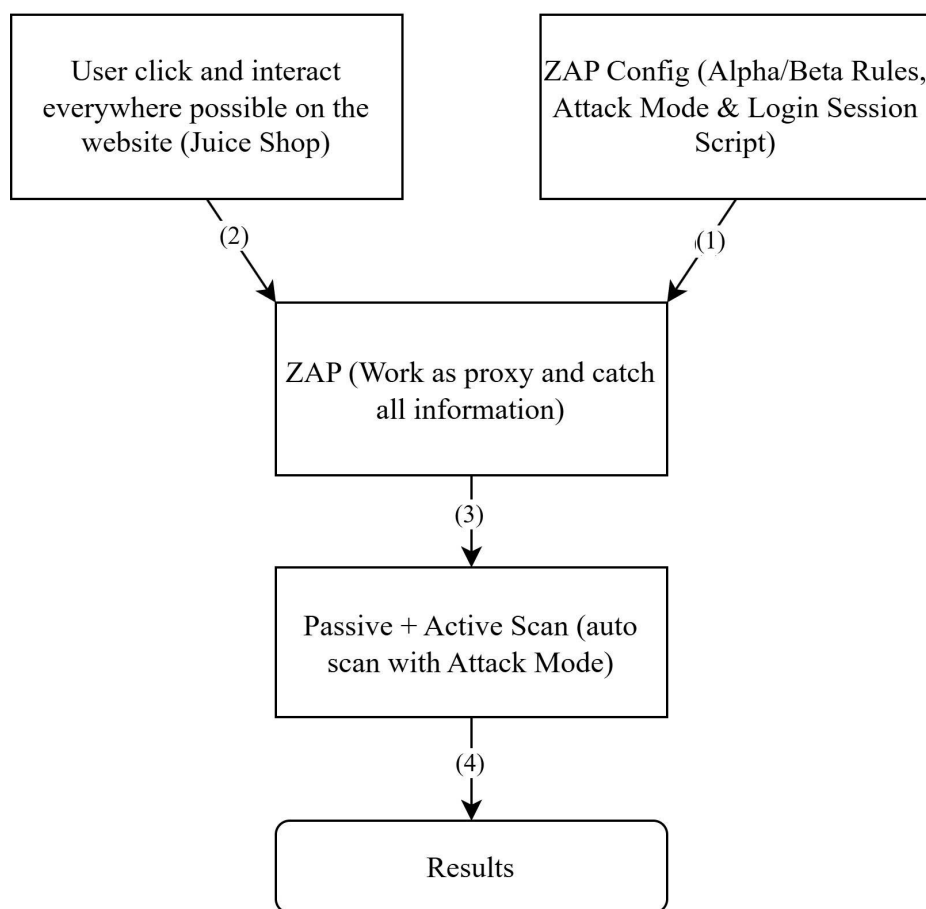
3.2.1 Phương pháp xây dựng site-map đầy đủ cho ứng dụng Web SPA

Phương pháp thuộc hướng tiếp cận thứ nhất được trình bày trong phần này tập trung khắc phục hạn chế của công cụ ZAP trong việc xây dựng site-map khi kiểm thử các ứng dụng Web SPA. Phương pháp được triển khai qua hai giai đoạn: thủ công và tự động. Ở giai đoạn thủ công, người kiểm thử trực tiếp tương tác với ứng dụng để chứng minh tính hiệu quả của phương pháp, tạo tiền đề để triển khai giai đoạn tự động. Tiếp đó, giai đoạn tự động được phát triển nhằm tự động hóa toàn bộ quy trình kiểm thử, thông qua việc kết hợp giữa công cụ tương tác tự động UI và công cụ kiểm thử bảo mật ZAP. Cách tiếp cận này giúp hình thành một quy trình kiểm thử có khả năng tự động thu thập, ghi nhận và phân tích cấu trúc ứng dụng, qua đó giúp ZAP nâng cao hiệu quả quét lỗ hổng và đảm bảo tính nhất quán trong quá trình quét.

Phương pháp này được phát triển thông qua hai giai đoạn triển khai chính:

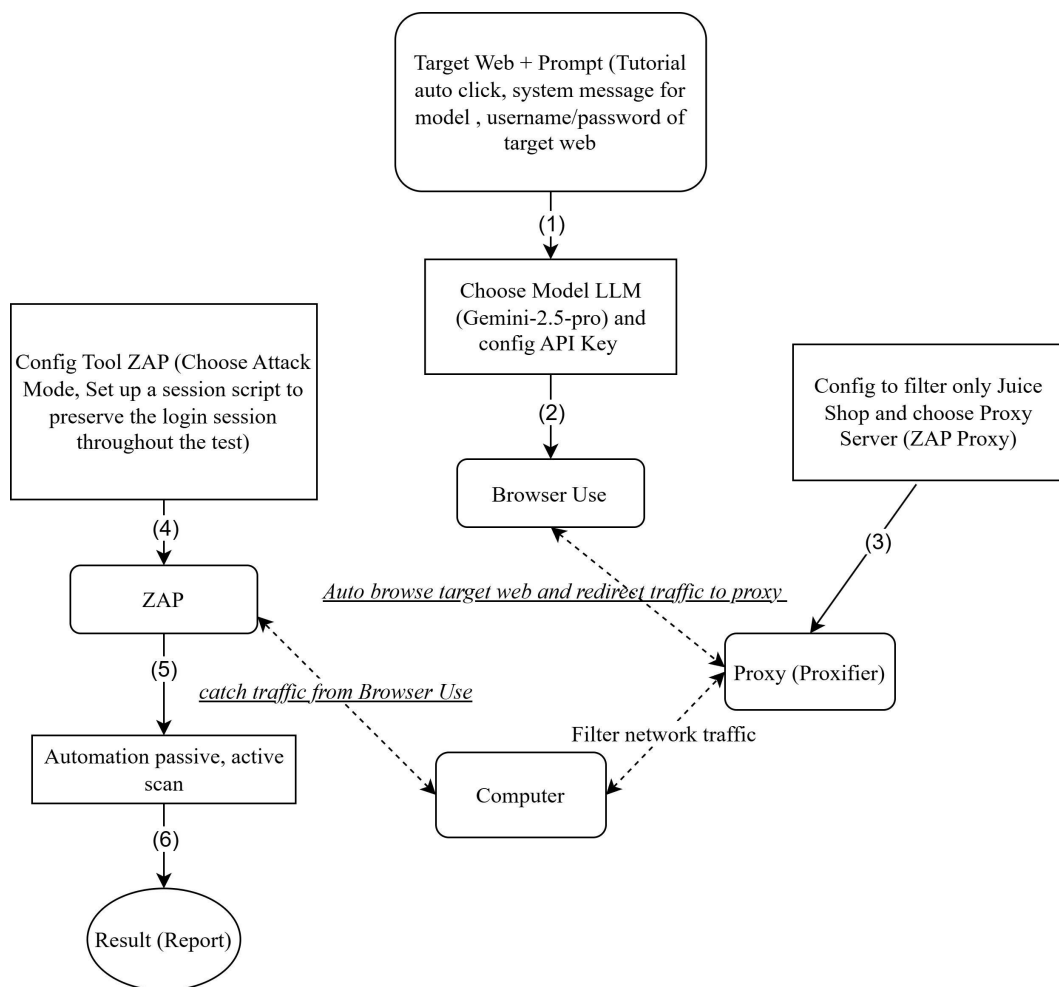
- **Giai đoạn một:** Xây dựng thủ công site-map cho ứng dụng.

Giai đoạn ban đầu bao gồm việc xây dựng thủ công sơ đồ trang hoàn chỉnh thông qua quá trình tương tác của người dùng với các ứng dụng Web SPA mục tiêu (ví dụ: OWASP Juice Shop). Phương pháp này được thực hiện nhằm chứng minh rằng việc xây dựng site-map toàn diện, giúp công cụ mở rộng phạm vi kiểm thử sẽ cải thiện đáng kể khả năng phát hiện lỗ hổng bảo mật.



Hình 3.1: Luồng kiểm thử bảo mật với phương pháp xây dựng site-map thủ công.

Hình 3.1 mô tả quy trình kiểm thử trong giai đoạn một được thực hiện bằng cách tương tác hoàn toàn thủ công nhằm mục đích xây dựng site-map và chứng minh tính hiệu quả khi công cụ có đủ URL. Nghiên cứu tiến hành tương tác trực tiếp với ứng dụng mục tiêu (ví dụ: OWASP Juice Shop [34]) thông qua các tương tác trên UI bao gồm nhấp chuột, di chuyển trong toàn bộ giao diện để kích hoạt mọi đường dẫn có thể có trong ứng dụng Web. Trong suốt quá trình này, ZAP hoạt động ở vai trò Proxy trung gian, ghi nhận toàn bộ yêu cầu và phản hồi giữa trình duyệt và máy chủ. Đồng thời, công cụ được cấu hình với các quy tắc quét Alpha, Beta và chế độ Attack Mode để tự động tiến hành quét chủ động mỗi khi có một tài nguyên mới được thu thập. Kết quả của quá trình này là có được tập hợp đầy đủ các đường dẫn của ứng dụng mục tiêu, mở rộng phạm vi kiểm thử và tăng khả năng phát hiện các lỗ hổng bảo mật.



Hình 3.2: Luồng kiểm thử và xây dựng site-map tự động.

- **Giai đoạn hai:** Phát triển quy trình tự động.

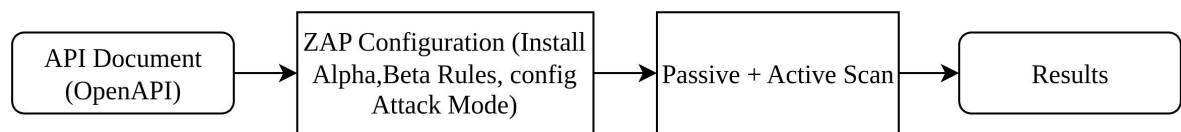
Sau khi kiểm chứng được tính hiệu quả của phương pháp thủ công, khóa luận tiếp tục đề xuất một giải pháp tự động hóa toàn diện bằng cách triển khai thêm công nghệ tự động hóa các tương tác UI là **Browser Use** kết hợp với ZAP để phục vụ cho mục đích kiểm thử bảo mật động. Giai đoạn này hướng tới việc xây dựng một quy trình kiểm thử tự động có khả năng mở rộng và tái sử dụng, qua đó khắc phục những hạn chế về thời gian và nhân lực vốn tồn tại trong quá trình xây dựng site-map thủ công. Nhờ vào cơ chế tự động hóa này, quá trình thu thập, ghi nhận và xây dựng site-map được thực hiện hoàn toàn tự động, giúp đảm bảo tính nhất quán, mở rộng phạm vi và nâng cao hiệu quả của quy trình kiểm thử bảo mật.

Hình 3.2 mô tả quy trình kiểm thử tự động hoạt động thông qua một chuỗi các bước nhằm tự động hóa toàn bộ quá trình xây dựng site-map và quét lỗ hổng bảo mật. Quy trình này bao gồm các bước chính như sau:

1. Hệ thống tiếp nhận các kịch bản tương tác Web được cung cấp bởi người kiểm thử, trong đó mô tả chi tiết các thao tác, điều hướng và kịch bản hành vi trên ứng dụng mục tiêu. Các thông tin này đóng vai trò là đầu vào cho quy trình kiểm thử, giúp mô hình ngôn ngữ hiểu và đưa ra quyết định cho quá trình tương tác thực tế.
2. Công nghệ **Browser Use** kết hợp với mô hình ngôn ngữ lớn **Gemini-2.5-pro** được sử dụng để tự động điều hướng và thao tác trên toàn bộ giao diện của ứng dụng. Quy trình này giúp thu thập toàn bộ các URL, qua đó giúp công cụ ZAP xây dựng site-map đầy đủ, làm cơ sở cho các giai đoạn quét bảo mật tiếp theo.
3. Trong quá trình tương tác tự động, toàn bộ lưu lượng truy cập được chuyển hướng và ghi vào vào máy chủ Proxy của ZAP, các thiết lập trên công cụ *Proxifier* giúp lọc và ghi nhận các yêu cầu đến từ ứng dụng mục tiêu cần kiểm thử.
4. Song song với quá trình tương tác tự động để thu thập URL, công cụ ZAP sử dụng site-map được xây dựng bởi các URL để thực hiện quét thụ động và quét chủ động, kết hợp cả hai quá trình giúp mở rộng phạm vi kiểm thử, nâng cao hiệu quả phát hiện lỗ hổng có trong ứng dụng.

5. ZAP tự động tổng hợp kết quả và sinh ra báo cáo chi tiết, liệt kê các điểm yếu bảo mật được phát hiện trên ngữ cảnh kịch bản tương tác.

Bên cạnh quá trình thu thập và xây dựng site-map thông qua các tương tác người dùng hoặc cơ chế tự động hóa giao diện, khóa luận cũng tận dụng các tài liệu đặc tả API của ứng dụng mục tiêu làm nguồn dữ liệu đầu vào bổ sung site-map cho công cụ kiểm thử. Các tài liệu này thường được cung cấp trong môi trường phát triển ứng dụng, mô tả chi tiết các điểm cuối API, tham số và luồng yêu cầu – phản hồi của hệ thống.



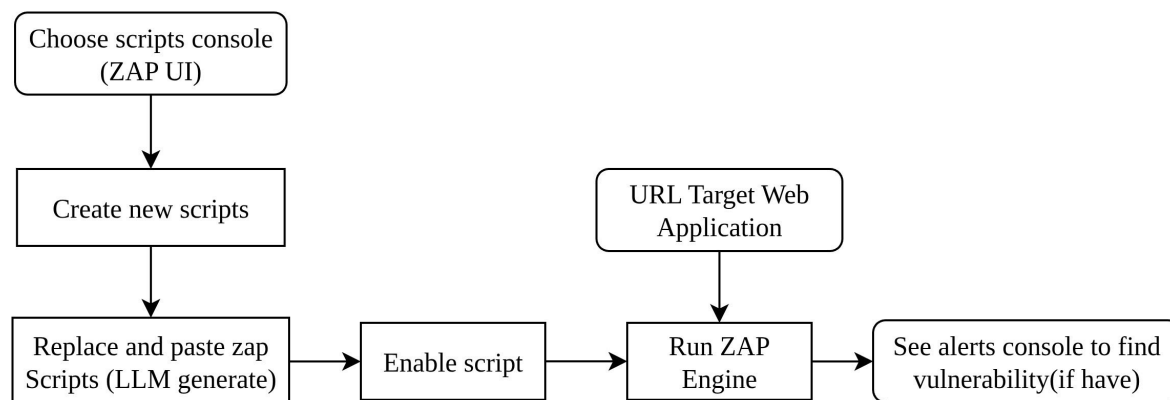
Hình 3.3: Luồng kiểm thử với tài liệu OpenAPI.

Tệp đặc tả giúp ZAP mở rộng site-map với đầy đủ các thông tin về yêu cầu, phản hồi và các điểm cuối API, qua đó mở rộng phạm vi phát hiện lỗ hổng sang các hệ thống Backend mà ứng dụng Web giao tiếp trong quá trình vận hành. Việc tích hợp dữ liệu từ tài liệu OpenAPI Swagger vào quy trình thu thập và xây dựng site-map, được xem là phần mở rộng của hướng tiếp cận thứ nhất, nhằm mở rộng phạm vi, cải thiện hiệu quả phát hiện lỗ hổng của công cụ trong quá trình kiểm thử bảo mật.

3.2.2 Phương pháp phát triển tập luật cho công cụ ZAP nhằm tăng cường khả năng phát hiện lỗ hổng

Hướng tiếp cận thứ hai xây dựng các luật kiểm thử bổ sung để phát hiện những lỗ hổng mà bộ luật quét mặc định của ZAP chưa bao phủ, đồng thời đề xuất triển khai hệ thống hỗ trợ tự động sinh luật kiểm thử bằng cách kết hợp mô hình ngôn ngữ lớn và các kỹ thuật điều chỉnh đầu ra của mô hình với dữ liệu bổ sung từ cơ sở tri thức chuyên ngành, giúp rút ngắn thời gian xây dựng các luật mới, nâng cao hiệu quả phát triển tập luật phát hiện lỗ hổng.

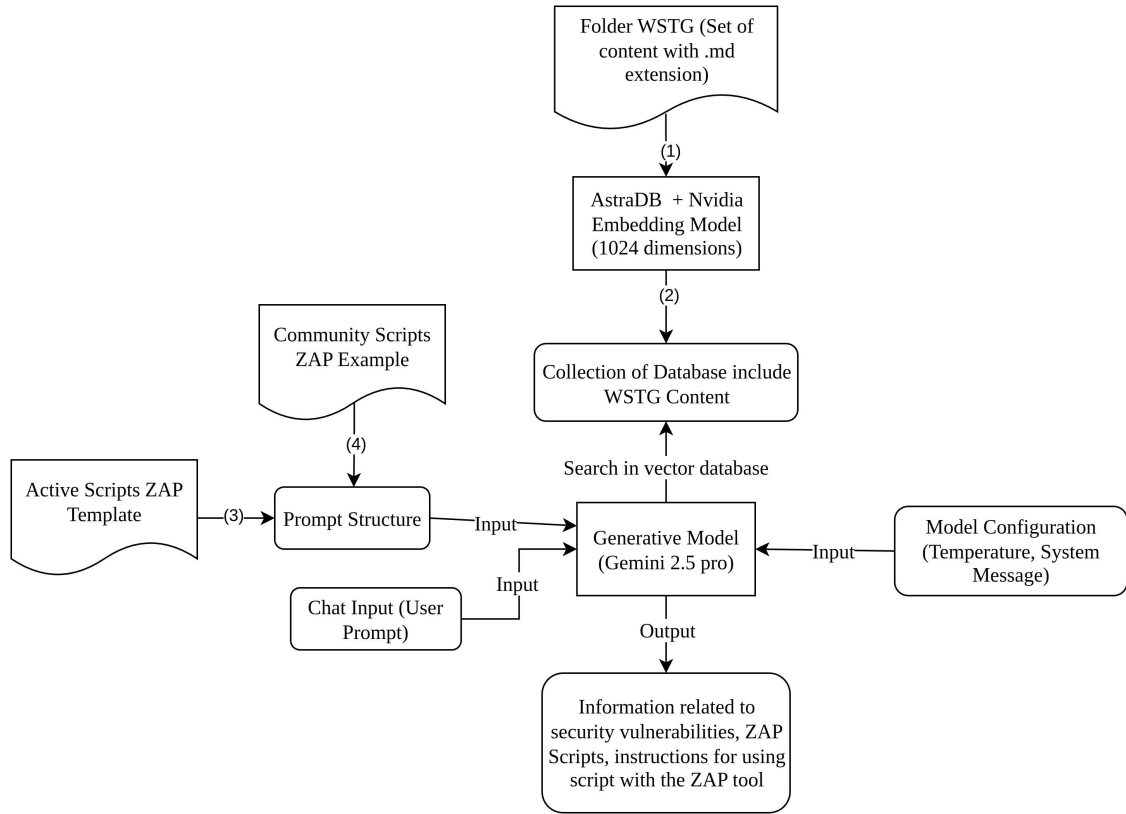
Các luật kiểm thử được tạo ra từ quy trình này có thể tích hợp trực tiếp vào lõi xử lý của công cụ ZAP, tận dụng các API sẵn có để mở rộng khả năng quét và phân tích. Hình 3.4 minh họa luồng sử dụng của các luật tùy chỉnh ZAP, mô tả cách thức các luật tùy chỉnh được tạo sinh, cài đặt, kích hoạt và đồng bộ hóa trong môi trường kiểm thử bảo mật và quy trình CI/CD.



Hình 3.4: Luồng hoạt động của ZAP với luật kiểm thử tùy chỉnh.

Hình 3.4 mô tả cách một luật kiểm thử tùy chỉnh được đưa vào ZAP và sử dụng trong quá trình quét. Người kiểm thử tạo một quy tắc quét mới trong giao diện của sổ script, sau đó dán mã luật đã sinh từ mô hình vào và kích hoạt để ZAP nạp luật mới. Khi quét ứng dụng đích, ZAP tự động áp dụng luật này cùng với các luật mặc định. Các kết quả quét cuối cùng được hiển thị trong mục cảnh báo của công cụ.

Mô hình ngôn ngữ lớn được sử dụng để hỗ trợ người kiểm thử trong việc sinh, chỉnh sửa và hoàn thiện các luật kiểm thử tùy chỉnh, qua đó rút ngắn thời gian phát triển mã luật và giảm sai sót về ngữ nghĩa, cú pháp khi làm việc trong môi trường GraalVM của ZAP. Trên cơ sở đó, khóa luận khảo sát và triển khai LLM với kỹ thuật RAG nhằm xây dựng hệ thống hỗ trợ tìm hiểu các thông tin về lỗ hổng bảo mật, hướng dẫn kiểm thử với các lỗ hổng phổ biến.



Hình 3.5: Kiến trúc RAG được thiết kế cho quá trình sinh luật kiểm thử ZAP.

Hình 3.5 minh họa kiến trúc của hệ thống RAG được thiết kế cho quá trình sinh tập luật ZAP. Hệ thống này lấy thông tin từ nhiều nguồn tri thức khác nhau, bao gồm tài liệu chính thức của OWASP về kiểm thử bảo mật ứng dụng Web (OWASP Web Security Testing Guide [32]), các ví dụ luật kiểm thử tùy chỉnh từ cộng đồng ZAP, cùng với các mẫu luật mặc định được cung cấp sẵn trong công cụ ZAP.

Trong quá trình triển khai giải pháp này, khóa luận sử dụng cơ sở dữ liệu vector trên nền tảng điện toán AstraDB cùng với mô hình nhúng từ vựng của Nvidia được cung cấp trong gói dùng thử. Sau khi hoàn thành bước này, hệ thống có được cơ sở dữ liệu chứa các hướng dẫn kiểm thử bảo mật trên ứng dụng Web. Khi được hỏi về lỗ hổng, quy trình đưa ra câu trả lời của hệ thống xử lý và tổng hợp đồng thời nhiều loại thông tin, bao gồm: khuôn mẫu của mã luật tương ứng với từng loại luật quét trong công cụ (ví dụ: Active Script, Passive Script, v.v), các ví dụ về luật kiểm thử được viết từ cộng đồng, prompt được viết theo cấu trúc của nền tảng Langflow và các tham số kỹ thuật của LLM để điều chỉnh kỳ vọng của câu trả lời. Hệ thống sau đó thực hiện truy vấn trên cơ sở dữ liệu vector để tìm ra

phương pháp phát hiện lỗ hổng phù hợp nhất với yêu cầu của người dùng. Cuối cùng, mô hình ngôn ngữ Gemini-2.5-Pro xử lý yêu cầu và sinh ra tập luật kiểm thử tương ứng với lỗ hổng, đảm bảo đúng cú pháp JavaScript và tuân theo khuôn mẫu của mã luật đã được cung cấp.

3.3 Tổng kết

Trong chương này, khóa luận đã trực tiếp trả lời cho câu hỏi nghiên cứu RQ1 bằng cách trình bày các hạn chế và đề xuất hai hướng tiếp cận chính nhằm nâng cao hiệu quả quy trình kiểm thử của công cụ OWASP ZAP. Hướng tiếp cận thứ nhất tập trung vào việc mở rộng khả năng khám phá ứng dụng, giúp ZAP thu thập và phân tích toàn diện hơn các thành phần, đường dẫn trong hệ thống kiểm thử mục tiêu. Hướng tiếp cận thứ hai tập trung vào việc phát triển thêm các tập luật kiểm thử, cho phép công cụ nhận diện được nhiều loại lỗ hổng bảo mật phức tạp hoặc gắn với logic nghiệp vụ đặc thù mà các quy tắc mặc định chưa bao quát. Sự kết hợp giữa hai hướng tiếp cận này tạo thành một quy trình kiểm thử bảo mật động toàn diện, vừa tăng cường khả năng bao phủ, vừa cải thiện độ chính xác trong việc phát hiện và phân loại lỗ hổng.

Trên cơ sở đó, chương tiếp theo sẽ trình bày chi tiết quá trình thực nghiệm được tiến hành nhằm đánh giá hiệu quả của phương pháp đã đề xuất, đồng thời phân tích các kết quả thu được để làm rõ những ưu điểm cũng như những hạn chế còn tồn tại của giải pháp.

Chương 4

Thực nghiệm và đánh giá

Chương này trình bày quá trình thực nghiệm và đánh giá giải pháp đề xuất, nhằm kiểm chứng tính hiệu quả và khả năng ứng dụng thực tiễn của phương pháp. Các thử nghiệm được triển khai trên các ứng dụng Web mục tiêu gồm OWASP Benchmark [35], OWASP Juice Shop [34] và Broken Crystal [36], đại diện cho cả mô hình ứng dụng Web truyền thống và ứng dụng Web SPA. Quy trình thực nghiệm bao gồm việc đánh giá khả năng phát hiện lỗ hổng của công cụ ZAP trước và sau khi áp dụng các cải tiến, qua đó so sánh, phân tích định tính và định lượng để xác định mức độ cải thiện về khả năng phát hiện lỗ hổng.

4.1 Mục tiêu thực nghiệm

Phần này giới thiệu các ứng dụng được lựa chọn làm mục tiêu kiểm thử, đóng vai trò là môi trường thực nghiệm để đánh giá hiệu quả của các giải pháp cải tiến công cụ ZAP. Ba ứng dụng Web được sử dụng để thực nghiệm trong khóa luận bao gồm: OWASP Benchmark, OWASP Juice Shop và Broken Crystal.

Bảng 4.1: Số lượng ca kiểm thử theo từng loại lỗ hổng trong Benchmark v1.2

Vulnerability Area	Numbers of Tests in v1.2	CWE Number
Command Injection	251	78
Weak Cryptography	246	327
Weak Hashing	236	328
LDAP Injection	59	90
Path Traversal	268	22
Secure Cookie Flag	67	614
SQL Injection	504	89
Trust Boundary Violation	126	501
Weak Randomness	493	330
XPATH Injection	35	643
XSS (Cross-Site Scripting)	455	79
Total Test Cases	2,740	

OWASP Benchmark [35] là bộ kiểm thử tiêu chuẩn được thiết kế chuyên biệt cho việc đánh giá công cụ kiểm thử bảo mật. Ứng dụng Web này hoạt động theo mô hình Web truyền thống (Server-side rendering), cung cấp các trường hợp kiểm thử được phân loại theo từng nhóm lỗ hổng cụ thể. Bảng 4.1 thể hiện cấu trúc rõ ràng của OWASP Benchmark với số lượng và đầu ra kỳ vọng xác định chính xác kết quả đúng/sai của từng ca kiểm thử. OWASP Benchmark được sử dụng nhằm cung cấp cơ sở đánh giá định lượng, cho phép đo lường các chỉ số như độ chính xác, tỷ lệ bỏ sót và năng lực phát hiện lỗ hổng thực tế của công cụ kiểm thử.

Bảng 4.2: Các lỗ hổng bảo mật xuất hiện trong OWASP Juice Shop

Category	OWASP	CWE
Broken Access Control	A1:2021, API1:2019, API5:2019	CWE-22, CWE-285, CWE-639, CWE-918
Broken Anti-Automation	OWASP-AT-004, API4:2019, OAT-009, OAT-015, OAT-008	CWE-362
Broken Authentication	A7:2021, API2:2019, P6:2021	CWE-287, CWE-352
Cross Site Scripting (XSS)	A3:2021, A7:2017	CWE-79
Cryptographic Issues	A2:2021	CWE-326, CWE-327, CWE-328, CWE-950
Improper Input Validation	API6:2019	CWE-20
Injection	A3:2021, API8:2019, P1:2021	CWE-74, CWE-89
Insecure Deserialization	A8:2021, A8:2017	CWE-502
Miscellaneous	P5:2021	Không có mã CWE cụ thể
Security Misconfiguration	A5:2021, A9:2021, API7:2019, API9:2019, API10:2019	CWE-209
Security through Obscurity	A4:2021, P5:2021	CWE-656
Sensitive Data Exposure	A3:2017, API3:2019, OTG-CONFIG-004, P2:2021	CWE-200, CWE-530, CWE-548
Unvalidated Redirects	A10:2013	CWE-601
Vulnerable Components	A6:2021	CWE-829, CWE-506, CWE-1104
XML External Entities (XXE)	A5:2021, A4:2017	CWE-611

Chú thích: A1–A10: OWASP Top 10 2021; API1–API10: OWASP API Security Top 10 2019. P: OWASP Top 10 Privacy Risks; CWE: Common Weakness Enumeration.

Khác với OWASP Benchmark, OWASP Juice Shop [34] đại diện cho kiểu ứng dụng Web SPA. Đây là một ứng dụng thương mại điện tử giả lập được cố tình tích hợp hầu hết các lỗ hổng Top 10 OWASP, các lỗ hổng xuất hiện được thể hiện trong bảng 4.2. Bên cạnh đó, ứng dụng còn chứa nhiều luồng nghiệp vụ phức tạp như quy trình mua hàng, thanh toán, quản lý tài khoản hay xử lý giỏ hàng, đòi hỏi sự kết hợp của nhiều bước liên tiếp. Những đặc điểm này khiến công cụ ZAP gặp nhiều hạn chế trong việc thu thập URL xây dựng site-map. Vì vậy, ứng dụng này được lựa chọn làm đối tượng đánh giá cho các phương pháp được đề xuất trong khóa luận, phản ánh được đặc trưng của các ứng dụng Web SPA và những thách thức mà công cụ kiểm thử bảo mật gặp phải.

Ứng dụng kiểm thử thứ ba là **Broken Crystal**, một ứng dụng được phát triển theo mô hình SPA kết hợp RestfulAPI, với kiến trúc phía máy chủ dựa trên Node.js và giao diện người dùng xây dựng bằng React JS. Ứng dụng này được thiết kế như một hệ thống thương mại điện tử giả lập. Tương tự như OWASP Juice Shop, Broken Crystal tích hợp có chủ đích nhiều lỗ hổng bảo mật phổ biến để phục vụ cho mục tiêu đánh giá khả năng của các công cụ kiểm thử bảo mật. Khác với OWASP Juice Shop, Broken Crystal không cung cấp tài liệu ánh xạ chính thức giữa các lỗ hổng và danh mục chuẩn như CWE hay Top 10 OWASP. Trong quá trình nghiên cứu, khóa luận đã tiến hành phân tích tài liệu mô tả lỗ hổng, và tự xây dựng ánh xạ thủ công giữa các lỗ hổng xuất hiện trên ứng dụng thành các mã CWE. Mục tiêu của việc này là tạo ra một cơ sở để có thể so sánh định lượng và đánh giá hiệu quả phát hiện của công cụ ZAP khi áp dụng các phương pháp cải tiến được đề xuất. Các mã CWE được ghi nhận trong quá trình ánh xạ bao gồm:

CWE-20, CWE-22, CWE-74, CWE-78, CWE-79, CWE-89, CWE-90, CWE-93, CWE-94, CWE-200, CWE-201, CWE-209, CWE-284, CWE-285, CWE-307, CWE-347, CWE-352, CWE-362, CWE-400, CWE-425, CWE-434, CWE-502, CWE-528, CWE-538, CWE-548, CWE-611, CWE-614, CWE-639, CWE-643, CWE-693, CWE-749, CWE-798, CWE-840, CWE-915, CWE-918, CWE-1004, CWE-1104, CWE-116, CWE-1321, và CWE-829.

Đáng chú ý, trong số này có nhiều mã thuộc nhóm **CWE Top 25 Most Dangerous Software Weaknesses (2024)**, tức là những điểm yếu phần mềm phổ biến và có mức độ ảnh hưởng nghiêm trọng nhất theo đánh giá của MITRE. Cụ thể là các mã: CWE-20, CWE-22, CWE-78, CWE-79, CWE-89, CWE-94, CWE-200, CWE-352, CWE-434, CWE-918, CWE-798, và CWE-400.

Với phạm vi lỗ hổng bao phủ rộng đối với các nhóm Top 25 CWE, cùng các công nghệ mới là Node.js và ReactJS được sử dụng nhiều trong ứng dụng Web thực tế hiện nay. Vì lý do đó, **Broken Crystal** được sử dụng trong khóa luận này với vai trò là ứng dụng kiểm thử mục tiêu để đánh giá hiệu quả của các phương pháp cải tiến công cụ ZAP được đề xuất.

4.2 Quy trình thực nghiệm

Trong phần này, khóa luận sẽ trình bày chi tiết quy trình thực nghiệm nhằm kiểm chứng hiệu quả của giải pháp kiểm thử bảo mật được đề xuất. Quy trình bao gồm việc tiến hành kiểm thử trên các ứng dụng mục tiêu với hai giai đoạn:

- **Giai đoạn 1:** Đánh giá khả năng phát hiện lỗ hổng của công cụ trong trạng thái mặc định
- **Giai đoạn 2:** Đánh giá khả năng phát hiện lỗ hổng của công cụ sau khi kết hợp với giải pháp cải tiến được đề xuất

Trước tiên, các tiêu chí đo lường, phương pháp đánh giá và các luật kiểm thử được cài đặt bổ sung sẽ được trình bày để làm cơ sở cho việc phân tích và so sánh kết quả thực nghiệm ở các mục tiếp theo.

4.2.1 Thông số đánh giá

Trong chương này, với trang web mục tiêu OWASP Benchmark, giải pháp được đánh giá độ hiệu quả dựa trên nhiều thông số định lượng, phản ánh một cách khoa học và chính xác về khả năng phát hiện lỗ hổng của công cụ kiểm thử bảo mật:

- **True Positive (TP):** Số lượng trường hợp lỗ hổng thực sự tồn tại và đã được công cụ phát hiện chính xác. Thông số này thể hiện năng lực nhận diện đúng các lỗ hổng bảo mật.
- **True Negative (TN):** Số lượng trường hợp không tồn tại lỗ hổng và công cụ cũng không báo động. Giá trị TN cao cho thấy khả năng phân biệt chính xác giữa ứng dụng an toàn và ứng dụng có lỗ hổng.

- **False Positive (FP):** Số lượng trường hợp công cụ cảnh báo có lỗ hổng nhưng thực tế lại không tồn tại. Đây là yếu tố làm gia tăng chi phí kiểm thử do mất thời gian xử lý cảnh báo sai.
- **False Negative (FN):** Số lượng trường hợp lỗ hổng tồn tại nhưng công cụ không phát hiện được. Đây là chỉ số nguy hiểm vì phản ánh các lỗ hổng bị bỏ sót, dễ gây ra rủi ro bảo mật trong thực tế.
- **Accuracy:** Tỷ lệ trung bình thể hiện mức độ đúng đắn tổng thể của công cụ trên toàn bộ tập kiểm thử, tính dựa trên cả TP và TN:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

- **Precision:** Tỷ lệ lỗ hổng được công cụ báo cáo là đúng trong số tất cả các cảnh báo:

$$Precision = \frac{TP}{TP + FP}$$

- **Recall:** Tỷ lệ lỗ hổng được công cụ phát hiện thành công trong số tất cả các lỗ hổng thực sự tồn tại:

$$Recall = \frac{TP}{TP + FN}$$

- **False Positive Rate (FPR):** Tỷ lệ cảnh báo sai trên tổng số trường hợp không có lỗ hổng:

$$FPR = \frac{FP}{FP + TN}$$

Còn đối với trang web mục tiêu OWASP Juice Shop và Broken Crystal, do hạn chế về tài liệu và mô tả ánh xạ chi tiết cho từng trường hợp kiểm thử, khóa luận chỉ tiến hành đánh giá dựa trên hai tiêu chí định tính:

1. Các lỗ hổng kỳ vọng đã phát hiện được.
2. Các lỗ hổng kỳ vọng chưa phát hiện được.

Tiêu chí đánh giá cuối cùng được bổ sung là đánh giá về kết quả đầu ra của LLM với các luật kiểm thử được tạo sinh cho công cụ ZAP, tập trung xem xét mức độ tuân thủ về mặt cấu trúc trong câu trả lời của mô hình, tính chính xác và khả năng thực thi của các luật khi tích hợp vào công cụ kiểm thử bảo mật. Đồng thời,

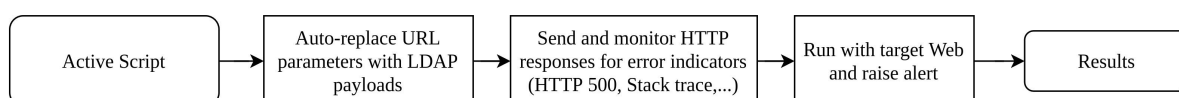
tiêu chí này cũng mở rộng sang việc đánh giá hiệu quả của công cụ tự động hóa UI dựa trên kịch bản ngôn ngữ tự nhiên, thông qua việc kiểm tra độ chính xác trong các thao tác tự động và khả năng thực hiện đầy đủ các yêu cầu dựa trên các kịch bản đầu vào, nhằm xác định mức độ ổn định và tin cậy của quá trình tự động.

4.2.2 Luật kiểm thử bổ sung

Để làm rõ hơn quy trình vận hành và khả năng áp dụng thực tế của phương pháp, phần này trình bày các thực nghiệm chi tiết về các luật tùy chỉnh được xây dựng và triển khai thử nghiệm trong môi trường ZAP, tương ứng với các nhóm lỗ hổng bảo mật phổ biến là LDAP Injection (A03-TOP 10 OWASP), CWE-200 (Information Exposure), CWE-352 (Cross-Site Request Forgery) và CWE-93 (CRLF Injection). Các luật thực nghiệm được nêu ra đóng vai trò minh họa cho quy trình tìm hiểu và sinh luật, cách thức tích hợp luật vào công cụ và cơ chế hoạt động của từng luật, điều này cho thấy cách thức các phương pháp đề xuất được triển khai trong môi trường kiểm thử:

• Thực nghiệm số 1 – Luật tùy chỉnh để phát hiện lỗ hổng LDAP Injection(A03-TOP 10 OWASP):

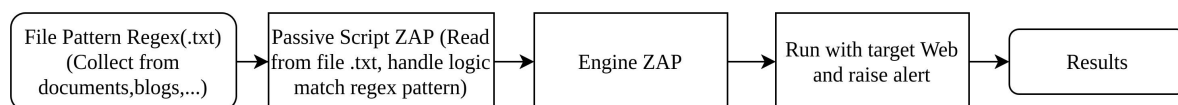
Trong công cụ kiểm thử bảo mật động ZAP, mặc dù tồn tại luật tấn công nhằm phát hiện LDAP Injection, nhưng tập hợp các payload và các mẫu nhận diện lỗ hổng đi kèm còn hạn chế. Vì vậy, khóa luận xây dựng luật kiểm thử tùy chỉnh để bổ sung thêm payload kiểm thử và mở rộng tập các mẫu nhận diện phản hồi nhằm tăng độ bao phủ và chính xác của công cụ ZAP trong phát hiện LDAP Injection.



Hình 4.1: Minh họa luồng hoạt động của luật tùy chỉnh cho lỗ hổng LDAP Injection.

Luồng hoạt động cụ thể được minh họa trong hình 4.1. Khi quét, công cụ thực hiện thay thế các tham số từ URL bằng payload đã định nghĩa, gửi yêu cầu đến máy chủ, và phân tích phản hồi để tìm kiếm các mẫu lỗi đã được định nghĩa trước (VD: HTTP 500, Stack trace error,v.v). Nếu phát hiện khớp mẫu, ZAP sinh cảnh báo kèm thông tin chi tiết về tham số, payload và vị trí phát hiện lỗ hổng.

- **Thực nghiệm số 2 – Luật tùy chỉnh để phát hiện lỗ hổng Information Exposure (CWE-200):**

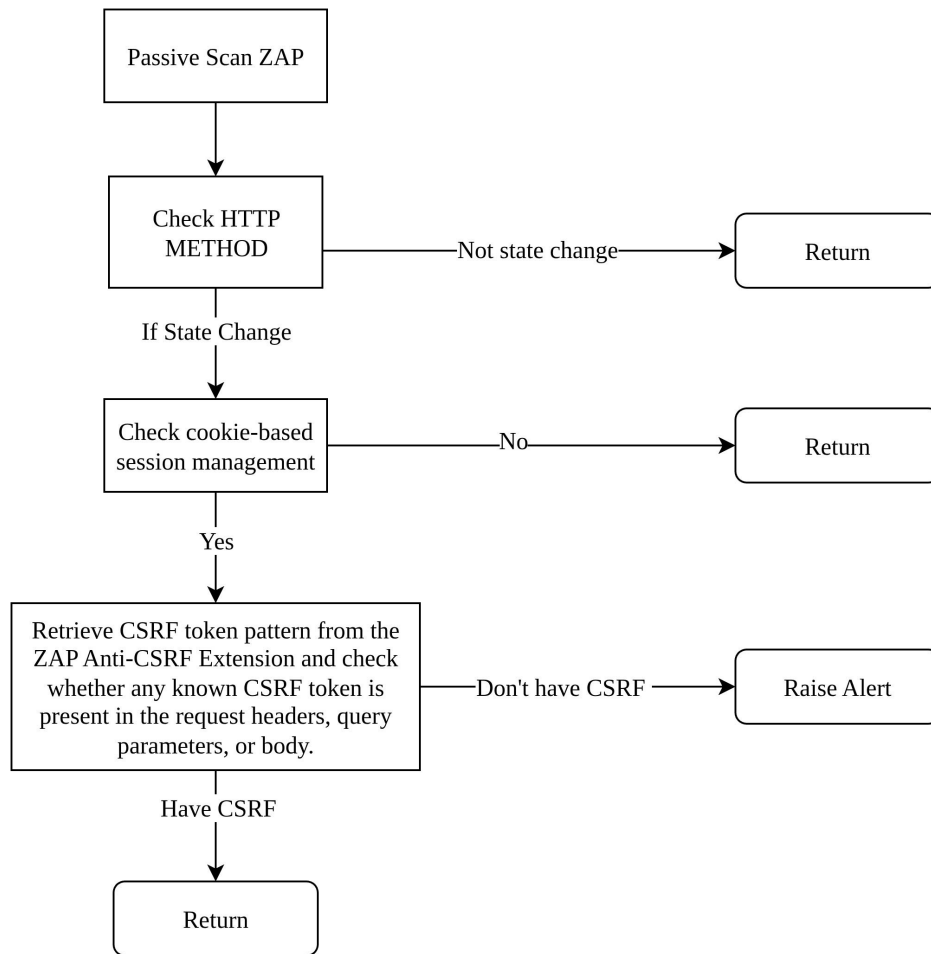


Hình 4.2: Minh hoạ luồng hoạt động của luật tùy chỉnh CWE-200.

Luồng hoạt động của luật kiểm thử tùy chỉnh CWE-200 được minh họa trong Hình 4.2. Ở bước đầu tiên, cần thu thập và tổng hợp các mẫu biểu thức chính quy từ nhiều nguồn tri thức khác nhau như tài liệu kỹ thuật, blog bảo mật hoặc cơ sở dữ liệu lỗ hổng, đồng thời các mẫu biểu thức chính quy này cũng có thể tự tùy chỉnh để theo nhu cầu kiểm thử của từng hệ thống. Sau đó biểu thức chính quy này được lưu trữ trong tệp định dạng `.txt`, hoặc có thể thay thế bằng các dạng lưu trữ khác. Các mẫu biểu thức này là dấu hiệu nhận biết rò rỉ thông tin nhạy cảm thường gặp trong phản hồi HTTP, chẳng hạn như khóa truy cập, chuỗi cấu hình hệ thống hoặc đường dẫn nội bộ. Trong bước kế tiếp, luật quét thụ động của ZAP được thiết lập để công cụ đọc các biểu thức chính quy, xử lý logic đối sánh. Khi quá trình quét được thực thi với trang web mục tiêu, ZAP sẽ thụ động phân tích các yêu cầu và phản hồi HTTP, kiểm tra nội dung của chúng dựa trên các mẫu đã định nghĩa, và tự động tạo cảnh báo nếu phát hiện thông tin nhạy cảm. Kết quả quét được hiển thị trực tiếp trên giao diện ZAP, bao gồm thông tin chi tiết về vị trí, loại dữ liệu bị lộ và mức độ rủi ro.

- **Thực nghiệm số 3 – Luật tùy chỉnh để phát hiện lỗ hổng Cross-Site Request Forgery (CWE-352):**

Trong phiên bản mặc định, OWASP ZAP đã được trang bị một số quy tắc kiểm thử nhằm phát hiện các biểu mẫu trong Form HTML thiếu cơ chế `CSRF Token`. Tuy nhiên, cơ chế này chỉ hiệu quả đối với các ứng dụng truyền thống có phản hồi ở dạng biểu mẫu. Đối với các ứng dụng SPA hiện nay sử dụng cơ chế giao tiếp thông qua RestfulAPI, chẳng hạn như ứng dụng `Broken Crystals` [36], phản hồi của máy chủ không chứa Form HTML khiến luật kiểm thử mặc định không thể phát hiện được lỗ hổng. Do đó, khả năng phát hiện của ZAP đối với CWE-352 trong trường hợp này còn hạn chế.



Hình 4.3: Minh họa luồng hoạt động của luật tùy chỉnh CWE-352.

Hình 4.3 minh họa luồng hoạt động của luật kiểm thử tùy chỉnh được xây dựng nhằm phát hiện lỗ hổng CWE-352 trong quá trình kiểm thử bảo mật động. Trước hết, mã kiểm thử tùy chỉnh được khởi tạo dưới dạng luật quét thụ động trong ZAP, khi mỗi yêu cầu được gửi đến và được ghi lại trong ZAP, công cụ tiến hành kiểm tra thông tin HTTP tương ứng. Nếu yêu cầu không thuộc nhóm thao tác thay đổi trạng thái (chẳng hạn **GET**), quá trình phân tích sẽ dừng lại để tối ưu hiệu suất. Trong trường hợp yêu cầu sử dụng các phương thức **POST**, **PUT**, **DELETE** hoặc **PATCH** – tức là có khả năng làm thay đổi dữ liệu trên máy chủ – luật kiểm thử sẽ tiếp tục bước đánh giá tiếp theo.

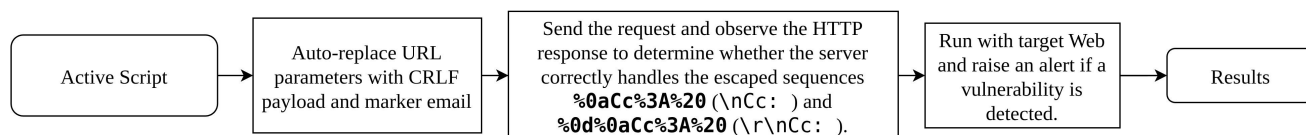
Ở bước kế tiếp, luật quét thụ động kiểm tra xem ứng dụng có đang sử dụng cơ chế xác thực dựa trên cookie-session hay không. Đây là yếu tố quan trọng vì CSRF chỉ xảy ra khi trình duyệt tự động gửi cookie xác thực hợp lệ cùng với yêu cầu. Nếu không phát hiện cookie xác thực, luật kiểm thử sẽ kết thúc tại đây. Trong trường

hợp cookie tồn tại, công cụ sẽ truy xuất mẫu định danh **CSRF Token** từ phần mở rộng của công cụ ZAP là "**Anti-CSRF Extension**", tiến hành tìm kiếm token này trong ba vị trí chính của yêu cầu: phần tiêu đề của yêu cầu, tham số truy vấn và nội dung thân. Nếu trong toàn bộ yêu cầu không tìm thấy bất kỳ **CSRF Token** nào, hoặc cookie không được gắn thuộc tính bảo vệ **SameSite**, luật kiểm thử sẽ tự động kích hoạt cảnh báo và ghi nhận kết quả vào báo cáo quét. Ngược lại, nếu phát hiện token hợp lệ hoặc cơ chế bảo vệ **SameSite** được thiết lập đúng, tập lệnh sẽ kết thúc mà không sinh cảnh báo.

Phương pháp này được thiết kế với mục tiêu tăng tính linh hoạt của công cụ ZAP trong việc phát hiện lỗ hổng CSRF. Thay vì chỉ dựa trên việc phân tích các Form HTML như ở phiên bản mặc định, luật kiểm thử tùy chỉnh cho phép đánh giá toàn bộ yêu cầu HTTP, bao gồm cả những yêu cầu gửi qua API. Nhờ đó, công cụ có thể nhận diện được nhiều kịch bản tấn công CSRF phức tạp hơn, đồng thời giảm thiểu tỷ lệ bỏ sót trong các ứng dụng web hiện đại.

● **Thực nghiệm số 4 – Luật tùy chỉnh để phát hiện lỗ hổng CWE-93 (CRLF Injection):**

Công cụ kiểm thử bảo mật động ZAP đã được trang bị các quy tắc mặc định để phát hiện CRLF Injection. ZAP thực hiện tiêm payload chứa các ký tự `\r\n` (`%0d%0a`) cùng một chuỗi đánh dấu vào HTTP header, sau đó quan sát phản hồi để xác định sự xuất hiện của chuỗi này. Tuy nhiên, cơ chế quét này chưa bao phủ được các kịch bản tấn công khác, chẳng hạn như trường hợp CRLF xảy ra trong các ứng dụng Webmail. Trong môi trường Webmail, bản chất các yêu cầu được gửi đi sẽ được xử lý bởi các giao thức email như SMTP/IMAP/POP3. Khi các ký tự này bị xử lý không đúng cách, kẻ tấn công có thể chèn thêm các header tùy chỉnh (như tiêu đề **To:**, **Cc:** hoặc **Bcc:**) vào hộp thư và chuyển hướng hộp thư chứa thông tin nhạy cảm đến địa chỉ của kẻ tấn công, gây ra rò rỉ dữ liệu nghiêm trọng. Do đó, khả năng phát hiện CRLF Injection của ZAP trong các kịch bản tương tác đa giao thức còn nhiều hạn chế.



Hình 4.4: Minh hoạ luồng hoạt động của luật tùy chỉnh CWE-93.

Hình 4.4 mô tả luồng hoạt động của luật kiểm thử tùy chỉnh được xây dựng nhằm phát hiện lỗ hổng CRLF Injection. Trước tiên, mã kiểm thử tùy chỉnh được khởi tạo dưới dạng luật quét chủ động. Khi luật được kích hoạt, công cụ sẽ tự động thay thế tất cả giá trị tham số trong URL của HTTP bằng chuỗi payload được thiết kế để chứa ký tự thoát CRLF (`\r` và `\n`) kèm theo một địa chỉ hộp thư đánh dấu để giúp nhận diện kết quả trong phản hồi từ máy chủ. Ở bước tiếp theo, yêu cầu cùng với payload sẽ được gửi đến máy chủ. Công cụ tiến hành quan sát và phân tích phản hồi HTTP để xác định cách máy chủ xử lý các chuỗi ký tự thoát `\r` và `\n`. Logic của quá trình này là kiểm tra xem chuỗi đánh dấu kèm các ký tự CRLF có xuất hiện trong phản hồi hay không, đồng thời xác nhận rằng yêu cầu được xử lý thành công với mã HTTP 200. Việc yêu cầu được xử lý một cách bình thường chứng tỏ rằng các ký tự `\r` và `\n` đã không được xử lý đúng cách, và bị diễn giải như ký tự xuống dòng, là dấu hiệu cho lỗ hổng CRLF trong xử lý Webmail.

4.2.3 Kiểm thử bảo mật trước khi cải tiến với khả năng mặc định của công cụ

Trong phạm vi thực nghiệm, các kết quả đều được thực hiện trên cùng một máy tính. Tất cả ca kiểm thử sử dụng cấu hình mặc định của công cụ và được chạy ở hai chế độ **Standard Mode** và **Attack Mode**. Khóa luận chưa xem xét các thiết lập nâng cao hay các cấu hình tùy chỉnh khác do hạn chế về tài nguyên phần cứng, đặc biệt là khả năng xử lý của CPU, khiến việc đánh giá trên các cấu hình khác nhau không khả thi trong giai đoạn này.

Bảng 4.3: Kết quả đánh giá theo từng loại lỗ hổng của công cụ ZAP trên trang Web OWASP Benchmark

Category	TP	TN	FP	FN	Accuracy	Precision	Recall	FPR	OWASP Top 10
cmdi	69	125	0	57	77.29%	100.00%	54.76%	0.00%	A03: Injection
crypto	0	116	0	130	47.15%	0.00%	0.00%	0.00%	A02: Cryptographic Failures
hash	0	107	0	129	45.34%	0.00%	0.00%	0.00%	A02: Cryptographic Failures
ldapi	0	32	0	27	54.24%	0.00%	0.00%	0.00%	A03: Injection
pathtraver	17	135	0	116	56.72%	100.00%	12.78%	0.00%	A01: Broken Access Control
securecookie	36	30	1	0	98.51%	97.30%	100.00%	3.23%	A05: Security Misconfiguration
sqli	161	222	10	111	75.99%	94.15%	59.19%	4.31%	A03: Injection
trustbound	0	43	0	83	34.13%	0.00%	0.00%	0.00%	A04: Insecure Design
weakrand	3	272	3	215	55.78%	50.00%	1.38%	1.09%	A02: Cryptographic Failures
xpathi	0	20	0	15	57.14%	0.00%	0.00%	0.00%	A03: Injection
xss	137	209	0	109	76.04%	100.00%	55.69%	0.00%	A03: Injection

Bảng 4.3 trình bày kết quả đánh giá chi tiết theo từng loại lỗ hổng bảo mật khi sử dụng công cụ ZAP trên bộ dữ liệu OWASP Benchmark. Nhìn chung, công cụ đạt hiệu quả tốt đối với một số nhóm lỗ hổng phổ biến như Command Injection, SQL Injection và XSS, thể hiện qua các chỉ số Precision và Accuracy cao (trên 75%, có trường hợp đạt tới 100%). Nhóm Secure Cookie Flag cũng đạt độ chính xác gần như tuyệt đối. Tuy nhiên, nhiều loại lỗ hổng khác như Weak Cryptography, Weak Hashing, LDAP Injection, Trust Boundary Violation hay XPath Injection lại có Recall bằng 0% hoặc rất thấp, cho thấy công cụ gần như bỏ sót hoàn toàn. Nguyên nhân là do một số lỗ hổng thuộc phạm vi kiểm thử tĩnh (ví dụ: Weak Cryptography, Weak Hashing, Trust Boundary Violation) vốn không phù hợp với phương pháp kiểm thử động của ZAP. Đáng chú ý, với các lỗ hổng XPath Injection và LDAP Injection, mặc dù thuộc phạm vi kiểm thử động và ZAP có luật kiểm thử, nhưng kết quả cho thấy các luật này chưa phát hiện được lỗ hổng. Vấn đề này chính là trọng tâm mà giải pháp được đề xuất trong khóa luận hướng đến xử lý. Ngoài ra, mặc dù tỷ lệ False Positive nhìn chung ở mức thấp, song một số trường hợp như SQL Injection và Secure Cookie Flag vẫn ghi nhận cảnh báo giả, phản ánh hạn chế nhất định trong độ tin cậy của công cụ.

Bảng 4.4: Khả năng phát hiện các CWE trong OWASP Juice Shop bằng công cụ ZAP

Category	CWE	Khả năng phát hiện bởi ZAP
Broken Access Control	CWE-22, CWE-285, CWE-639, CWE-918	Không phát hiện
Broken Anti-Automation	CWE-362	Không phát hiện
Broken Authentication	CWE-287, CWE-352	Có thể phát hiện (CWE-352)
Cross Site Scripting (XSS)	CWE-79	Không phát hiện
Cryptographic Issues	CWE-326, CWE-327, CWE-328, CWE-950	Không phát hiện
Improper Input Validation	CWE-20	Không phát hiện
Injection	CWE-74, CWE-89	Có thể phát hiện (CWE-89)
Insecure Deserialization	CWE-502	Không phát hiện
Miscellaneous	-	Không phát hiện
Security Misconfiguration	CWE-209	Không phát hiện
Security through Obscurity	CWE-656	Không phát hiện
Sensitive Data Exposure	CWE-200, CWE-530, CWE-548	Không phát hiện
Unvalidated Redirects	CWE-601	Không phát hiện
Vulnerable Components	CWE-829, CWE-506, CWE-1104	Có thể phát hiện (CWE-829)
XML External Entities (XXE)	CWE-611	Không phát hiện

Kết quả trong bảng trên cho thấy công cụ ZAP chỉ có khả năng phát hiện được một số lỗ hổng nhất định như Injection (CWE-89), Broken Authentication (CWE-352) và Vulnerable Components (CWE-829), trong khi phần lớn các nhóm lỗ hổng khác đều không được nhận diện. Hạn chế này xuất phát từ cơ chế khám phá ứng dụng Web của ZAP vốn gặp khó khăn trên các ứng dụng hiện đại như OWASP Juice Shop, nơi cấu trúc SPA và cơ chế tải động nội dung bằng JavaScript làm cho quá trình xây dựng bản đồ ứng dụng gặp khó khăn. Điều này dẫn đến việc nhiều lỗ hổng gắn với logic nghiệp vụ cần nhiều bước tương tác bị bỏ sót. Có thể thấy rằng với cấu hình mặc định, ZAP chưa đáp ứng được yêu cầu kiểm thử bảo mật toàn diện cho các ứng dụng Web hiện đại.

Cuối cùng, đối với ứng dụng Web mục tiêu Broken Crystal, do ứng dụng không cung cấp tài liệu ánh xạ trực tiếp giữa các lỗ hổng và chuẩn CWE, việc đánh giá được thực hiện dựa trên tài liệu CWE của MITRE, trong đó tập trung chủ yếu vào nhóm Top 25 CWE. Khác với Top 10 OWASP, tài liệu CWE mô tả chi tiết và chuyên biệt cho từng loại lỗ hổng, do đó phù hợp để làm cơ sở đối chiếu với tài liệu mô tả hạn chế của ứng dụng Broken Crystal. Kết quả nhận được ban đầu cho thấy công cụ ZAP phát hiện được các lỗ hổng: CWE-601, CWE-22, CWE-78, CWE-94, CWE-918, CWE-639, CWE-538, CWE-1004 và CWE-693. Khi ánh xạ sang Top 25 CWE, công cụ bao phủ 4/25 lỗ hổng phổ biến là CWE-22, CWE-78, CWE-94 và CWE-918. Trong Broken Crystal có chứa tổng số 12 lỗ hổng thuộc Top 25 CWE và nằm

trong phạm vi của DAST, công cụ ZAP phát hiện được 4 lỗ hổng, tương ứng với tỷ lệ bao phủ 33.33%. Những kết quả này cho thấy khả năng phát hiện tự động của ZAP còn hạn chế khi áp dụng trên các ứng dụng Web hiện đại. Đây là cơ sở để đánh giá các cải tiến được đề xuất, xác định mức độ nâng cấp về khả năng nhận diện lỗ hổng của công cụ sau khi áp dụng giải pháp.

4.2.4 Kiểm thử bảo mật sau khi áp dụng các phương pháp kiểm thử cải tiến

Phần này của khóa luận tập trung đánh giá hiệu quả của các giải pháp cải tiến được áp dụng cho công cụ kiểm thử bảo mật ZAP trên ba mục tiêu thử nghiệm: OWASP Juice Shop, Broken Crystal và OWASP Benchmark. Trong đó, hai ứng dụng đại diện cho mô hình Web SPA là OWASP Juice Shop và Broken Crystal được sử dụng để đánh giá mức độ cải thiện khả năng phát hiện lỗ hổng khi công cụ được bổ sung site-map cùng với các luật kiểm thử tùy chỉnh. Ứng dụng OWASP Benchmark đóng vai trò đánh giá định tính đối với các luật kiểm thử bổ sung, dựa trên test suite và các bài test case có sẵn để quan sát các thông số trước và sau khi áp dụng các luật mở rộng.

Trước hết, nội dung đánh giá phân tích hiệu quả phát hiện lỗ hổng của phương pháp mở rộng site-map, được thực hiện thông qua các triển khai chính: bổ sung đầu vào cho công cụ bằng tài liệu OpenAPI Swagger đối với các ứng dụng có tài liệu, tương tác thủ công và áp dụng pipeline tương tác tự động. Các cách tiếp cận này được triển khai trên hai ứng dụng là OWASP Juice Shop và Broken Crystal.

Khả năng phát hiện lỗ hổng bảo mật của hai phương pháp tương tác thủ công và sử dụng pipeline tự động là tương đương nhau. Tuy nhiên, việc mở rộng site-map thông qua tương tác thủ công hoàn toàn dựa vào thao tác của người kiểm thử, đòi hỏi tương tác đầy đủ các ca sử dụng ứng dụng Web giống như một người dùng cuối. Cách tiếp cận này tiềm ẩn nguy cơ bỏ sót các bước quan trọng trên những ứng dụng có logic phức tạp, tiêu tốn đáng kể thời gian và công sức để rà soát thủ công toàn bộ kịch bản ca sử dụng. Phương pháp sử dụng tài liệu OpenAPI Swagger chỉ hỗ trợ bổ sung site-map cho công cụ trên những hệ thống có tài liệu đầy đủ. Đồng thời, tài liệu API thường chỉ liệt kê các điểm cuối API giao tiếp với hệ thống Backend và có thể bỏ sót các URL phát sinh trong khi tương tác phía client.

Từ những hạn chế của phương pháp thủ công và phương pháp bổ sung tài liệu đặc tả API, khóa luận triển khai pipeline tương tác tự động UI với tác tử AI. Đầu vào của quy trình tự động là mô tả bằng ngôn ngữ tự nhiên các thao tác và kịch bản sử dụng ứng dụng Web. Pipeline này được thiết kế nhằm kiểm chứng khả năng thay thế cho tương tác thủ công, giúp giảm thiểu sai sót chủ quan, duy trì tính nhất quán giữa các lần thực thi và cho phép tái sử dụng các kịch bản ngôn ngữ tự nhiên qua nhiều lần kiểm thử.

Bảng 4.5: Kết quả phát hiện lỗ hổng trên Broken Crystal sau khi bổ sung site-map

ID	Before Improvement	After Improvement
CWE-79	x	o
CWE-89	x	o
CWE-352	x	x
CWE-22	o	o
CWE-78	o	o
CWE-434	x	x
CWE-94	o	o
CWE-20	x	x
CWE-200	x	x
CWE-918	o	o
CWE-798	x	x
CWE-400	x	x

Chú thích: x – không phát hiện lỗ hổng; o – phát hiện lỗ hổng.

Dựa trên kết quả trong Bảng 4.5, việc bổ sung site-map với thông tin từ tài liệu API Swagger giúp ZAP mở rộng phạm vi kiểm thử và phát hiện thêm hai lỗ hổng thuộc Top 25 CWE là CWE-79 và CWE-89, hai lỗ hổng này không phát hiện trong lần quét mặc định. Như vậy, khi chỉ bổ sung site-map mà chưa áp dụng các luật kiểm thử tùy chỉnh, ZAP có thể phát hiện tổng cộng 6/25 lỗ hổng trong Top 25 CWE, tương đương với 6/12 của Top 25 CWE thuộc về phạm vi của DAST có trong Broken Crystal.

Bảng 4.6: Kết quả phát hiện lỗ hổng trên OWASP Juice Shop sau khi bổ sung site-map

Category	CWE	Khả năng phát hiện bởi ZAP
Broken Access Control	CWE-22, CWE-285, CWE-639, CWE-918	Có thể phát hiện (CWE-22)
Broken Anti-Automation	CWE-362	Không phát hiện
Broken Authentication	CWE-287, CWE-352	Có thể phát hiện (CWE-352)
Cross Site Scripting (XSS)	CWE-79	Có thể phát hiện (CWE-79)
Cryptographic Issues	CWE-326, CWE-327, CWE-328, CWE-950	Không phát hiện
Improper Input Validation	CWE-20	Không phát hiện
Injection	CWE-74, CWE-89	Có thể phát hiện (CWE-89)
Insecure Deserialization	CWE-502	Không phát hiện
Miscellaneous	-	Không phát hiện
Security Misconfiguration	CWE-209	Không phát hiện
Security through Obscurity	CWE-656	Không phát hiện
Sensitive Data Exposure	CWE-200, CWE-530, CWE-548	Không phát hiện
Unvalidated Redirects	CWE-601	Có thể phát hiện (CWE-601)
Vulnerable Components	CWE-829, CWE-506, CWE-1104	Có thể phát hiện (CWE-829)
XML External Entities (XXE)	CWE-611	Không phát hiện

Với kết quả trong Bảng 4.6, bổ sung thêm URL vào site-map đã giúp ZAP xác định được các dấu hiệu liên quan đến những nhóm lỗ hổng phổ biến và có mức độ nguy hiểm cao trong OWASP Juice Shop như Broken Access Control (CWE-22), Cross-Site Scripting (CWE-79) và Unvalidated Redirects (CWE-601), qua đó nâng độ bao phủ từ 3/15 lên 6/15 nhóm lỗ hổng. Kết quả này cho thấy các kỹ thuật mở rộng site-map là một bước quan trọng để nâng cao hiệu quả kiểm thử, đặc biệt trong bối cảnh các thuật toán khám phá mặc định hoạt động kém hiệu quả trên ứng dụng Web SPA.

Tiếp theo, nội dung đánh giá sẽ tập trung phân tích mức độ cải thiện của ZAP khi bổ sung các luật kiểm thử mới trên cả ba ứng dụng Web mục tiêu. Phương pháp đánh giá bao gồm phân tích định tính dựa trên các thông số phản ánh độ hiệu quả của luật kiểm thử bổ sung, kết hợp với đánh giá định lượng nhằm xác định trên từng ứng dụng tồn tại lỗ hổng thì luật cải tiến có phát hiện thêm được lỗ hổng đó hay không.

Bảng 4.7: Kết quả đánh giá phát hiện lỗ hổng LDAP Injection và XPath Injection của phương pháp trên OWASP Benchmark

Category	TP	TN	FP	FN	Accuracy	Precision	Recall	FPR
LDAP Injection	15	32	0	12	79.66%	100.00%	55.56%	0.00%
XPath Injection	6	20	0	9	74.29%	100.00%	40.00%	0.00%

Sau khi áp dụng các cải tiến, kết quả cho thấy ZAP đã tăng rõ rệt khả năng phát hiện hai lỗ hổng LDAP Injection và XPath Injection. Cụ thể, cả hai loại đều đạt thông số Precision tuyệt đối (100%), thông số Recall tăng từ 0% (trước cải tiến) lên lần lượt 55.56% đối với LDAP và 40.00% đối với XPath, thông số Accuracy cũng được cải thiện, đạt 79.66% cho LDAP và 74.29% cho XPath. So với giai đoạn trước, khi nhiều trường hợp bị bỏ sót, các con số sau khi cải tiến chứng tỏ việc bổ sung luật quét bảo mật đã giúp công cụ mở rộng khả năng phát hiện cho lỗ hổng LDAP Injection và XPath Injection.

Minh họa luật kiểm thử mở rộng trên công cụ ZAP cho lỗ hổng LDAP Injection

```
1 function scan(as, msg, param, value) {
2   const payloads = [
3     '*', ' ) (uid=*)', '*(*)&%|', '(objectClass=*)',
4     '|(|(cn=*)(sn=*)(gn=*))', '\\2A', '()', '(&(objectClass=*)(uid=admin))'
5   ];
6   const evidencePatterns = [
7     /LDAPException|Invalid LDAP syntax|SearchRequest|Filter pattern/i,
8     /javax\.naming\.directory|javax\.naming\.NamingException/i,
9     /LDAP search failed|Protocol error|LDAP error/i,
10    /Unbalanced Parentheses|Invalid filter/i,
11    /supplied argument is not a valid ldap filter/i,
12    /objectclass violation/i,
13    /invalid attribute syntax/i
14  ];
15  for (let i = 0; i < payloads.length; i++) {
16    const attack = payloads[i];
17    if (as.isStop()) return;
18    const newMsg = msg.cloneRequest(); as.setParam(newMsg,param,attack);
19    as.sendAndReceive(newMsg,false,false);
20    const responseBody = newMsg.getResponseBody().toString();
21    for (let j = 0; j < evidencePatterns.length; j++) {
22      const match = evidencePatterns[j].exec(responseBody);
23      if (match) {
24        as.newAlert().setParam(param).setAttack(attack)
25          .setEvidence(match[0]).setOtherInfo("...").setMessage(newMsg).raise();
26        return;
27      }
28    }
29    if (newMsg.getResponseHeader().getStatusCode()===500) {
30      as.newAlert().setParam(param).setAttack(attack)
31        .setEvidence("Status code 500").setOtherInfo("...").setMessage(newMsg).raise();
32      return;
33    }
34  }
35 }
```

Trong quá trình tạo ra luật kiểm thử, khóa luận sử dụng hệ thống hỏi đáp RAG được triển khai với mục tiêu hỗ trợ viết script kiểm thử cho hai lỗ hổng LDAP Injection và XPath Injection. Hệ thống RAG đáp ứng cơ bản yêu cầu về cấu trúc câu trả lời, cung cấp đầy đủ các thông tin cần thiết liên quan đến lỗ hổng. Tuy nhiên, mã kiểm thử được sinh ra vẫn tồn tại một số sai lệch về cú pháp, cần được điều chỉnh thủ công để đảm bảo tương thích với engine của ZAP. Nhìn chung, trong hai luật kiểm thử được xây dựng cho LDAP Injection và XPath Injection, RAG đã hỗ trợ trong việc hình thành ý tưởng phát hiện, sinh các payload và dấu hiệu để nhận diện lỗ hổng, từ đó rút ngắn thời gian tiếp cận, thiết kế và phát triển luật kiểm thử.

Bảng 4.8: Kết quả phát hiện lỗ hổng trên OWASP Juice Shop sau khi bổ sung luật

Category	CWE	Khả năng phát hiện bởi ZAP
Broken Access Control	CWE-22, CWE-285, CWE-639, CWE-918	Có thể phát hiện (CWE-22)
Broken Anti-Automation	CWE-362	Không phát hiện
Broken Authentication	CWE-287, CWE-352	Có thể phát hiện (CWE-352)
Cross Site Scripting (XSS)	CWE-79	Có thể phát hiện (CWE-79)
Cryptographic Issues	CWE-326, CWE-327, CWE-328, CWE-950	Không phát hiện
Improper Input Validation	CWE-20	Không phát hiện
Injection	CWE-74, CWE-89	Có thể phát hiện (CWE-89)
Insecure Deserialization	CWE-502	Có thể phát hiện (CWE-502)
Miscellaneous	-	Không phát hiện
Security Misconfiguration	CWE-209	Không phát hiện
Security through Obscurity	CWE-656	Không phát hiện
Sensitive Data Exposure	CWE-200, CWE-530, CWE-548	Có thể phát hiện (CWE-200)
Unvalidated Redirects	CWE-601	Có thể phát hiện (CWE-601)
Vulnerable Components	CWE-829, CWE-506, CWE-1104	Có thể phát hiện (CWE-829)
XML External Entities (XXE)	CWE-611	Không phát hiện

Bảng kết quả 4.8 cho thấy việc bổ sung các luật tùy chỉnh đã góp phần nâng cao hiệu suất phát hiện của ZAP, cụ thể số lượng loại lỗ hổng được xác định tăng từ 6/15 lên 7/15. Các luật kiểm thử bảo mật phát triển thêm đã chứng minh hiệu quả, giúp công cụ phát hiện thành công lỗ hổng CWE-200 trong ứng dụng OWASP Juice Shop. Kết quả này khẳng định tính khả thi của việc bổ sung luật tùy chỉnh nhằm tăng khả năng phát hiện lỗ hổng của công cụ đối với ứng dụng Web SPA. Trong phạm vi nghiên cứu này, các đoạn mã mở rộng đối với các CWE còn lại trên OWASP Juice Shop không được đưa vào đánh giá chi tiết, bởi hai trang Web mục tiêu còn lại không chứa loại lỗ hổng tương tự.

Để tiếp tục đánh giá mức độ hiệu quả và tính khái quát hóa của các đoạn mã mở rộng, khóa luận triển khai kiểm thử với phương pháp và cấu hình tương đương trên ứng dụng mục tiêu là **Broken Crystal**. Mục đích là để xác thực rằng các cải tiến về mã kiểm thử không chỉ hiệu quả trên một môi trường mục tiêu ban đầu mà còn duy trì được khả năng phát hiện lỗ hổng khi áp dụng cho các ứng dụng khác.

Bảng 4.9: Kết quả phát hiện lỗ hổng trên Broken Crystal sau khi bổ sung mã kiểm thử

ID	Before Improvement	After Improvement
CWE-79	x	o
CWE-89	x	o
CWE-352	x	x
CWE-22	o	o
CWE-78	o	o
CWE-434	x	x
CWE-94	o	o
CWE-20	x	x
CWE-200	x	o
CWE-918	o	o
CWE-798	x	x
CWE-400	x	x
CWE-93	x	o
CWE-90	x	o

Chú thích: x – không phát hiện lỗ hổng; o – phát hiện lỗ hổng.

Các kết quả trong Bảng 4.9 cho thấy các luật tùy chỉnh được bổ sung trong khóa luận có khả năng áp dụng một cách tổng quát trên nhiều ứng dụng Web khác nhau. Cụ thể, luật bổ sung dành cho lỗ hổng CWE-90 (LDAP Injection) đã phát hiện thành công lỗ hổng này trên cả **OWASP Benchmark** và **Broken Crystal**. Tương tự, luật kiểm thử cho CWE-200 cũng cho thấy hiệu quả khi nhận diện được lỗ hổng trên cả hai ứng dụng Web SPA được đưa vào đánh giá. Bên cạnh đó, đối với **Broken Crystal**, khóa luận đã xây dựng thêm các luật nhằm khai thác các khía cạnh đối với lỗ hổng CWE-352 và CWE-93. Hai loại lỗ hổng này đều tồn tại trong Broken Crystal và đều có luật quét mặc định, tuy nhiên các luật mặc định không thể phát hiện được chúng. Sau khi bổ sung thêm các luật kiểm thử, số lỗ hổng được phát hiện trên ứng dụng **Broken Crystal** tăng từ 6/12 lên 9/12.

Tổng hợp các kết quả đánh giá đã chứng minh rằng các giải pháp cải tiến được đề xuất trong khóa luận – bao gồm mở rộng bản đồ ứng dụng thông qua bổ sung site-map và xây dựng các luật kiểm thử tùy chỉnh – đã đóng góp vào bài toán nâng cao hiệu quả phát hiện lỗ hổng của công cụ ZAP trên nhiều mô hình ứng dụng Web khác nhau. Các kết quả thực nghiệm chỉ ra phương pháp bổ sung URL vào site-map giúp ZAP mở rộng phạm vi kiểm thử, đặc biệt đối với các ứng dụng Web SPA có cấu trúc phức tạp mà cơ chế khám phá mặc định của công cụ khó xử lý đầy đủ. Đồng thời, các luật kiểm thử tùy chỉnh được xây dựng với sự hỗ trợ của hệ thống RAG đã chứng tỏ khả năng phát hiện lỗ hổng trong những tình huống tổng quát, không phụ thuộc vào đặc thù của từng ứng dụng cụ thể. Khi thử nghiệm các luật bổ sung trên OWASP Juice Shop, Broken Crystal và OWASP Benchmark thì các luật này đều có thể nhận diện được các lỗ hổng tương ứng khi chúng xuất hiện. Những kết quả này khẳng định rằng việc mở rộng bản đồ ứng dụng và bổ sung luật kiểm thử cho ZAP là một hướng tiếp cận phù hợp, đồng thời có tiềm năng phát triển thành các bộ luật kiểm thử tự động và chuyên sâu hơn trong tương lai.

4.3 Tổng kết

Chương này đã trình bày quy trình thực nghiệm và đánh giá khả năng phát hiện lỗ hổng của công cụ OWASP ZAP trên ba ứng dụng mục tiêu: OWASP Juice Shop, Broken Crystal và OWASP Benchmark. Các kết quả thu được ban đầu cho thấy năng lực của công cụ ở trạng thái mặc định, chỉ ra các hạn chế về khả năng bao phủ lỗ hổng, đặc biệt với những ứng dụng Web SPA. Sau khi áp dụng các giải pháp cải tiến, bao gồm mở rộng site-map bằng cách thủ công và tự động tương tác giao diện, cũng như bổ sung thêm luật kiểm thử, số lượng lỗ hổng được phát hiện tăng lên, bao gồm nhiều lỗ hổng thuộc Top 10 OWASP và Top 25 CWE. Kết quả khẳng định các giải pháp bổ sung góp phần tối ưu hiệu quả sử dụng của công cụ ZAP, nâng cao khả năng phát hiện lỗ hổng với các ứng dụng Web.

Chương 5

Kết luận

Khóa luận đã tập trung nghiên cứu và đề xuất các giải pháp nhằm nâng cao hiệu quả của công cụ kiểm thử bảo mật động đối với ứng dụng Web, với trọng tâm là công cụ OWASP ZAP. Dựa trên những hạn chế của công cụ, nghiên cứu đã đề xuất hai hướng cải tiến chính: xây dựng site-map đầy đủ cho ứng dụng Web SPA, mở rộng khả năng phát hiện lỗ hổng thông qua việc sinh thêm luật kiểm thử.

Kết quả thực nghiệm trên ba ứng dụng đại diện cho các kiểu ứng dụng Web phổ biến là OWASP Juice Shop, Broken Crystal và OWASP Benchmark đã chứng minh tính khả thi và hiệu quả của các giải pháp đề xuất. Công cụ ZAP sau khi được cải tiến đã thể hiện khả năng phát hiện tốt hơn đối với nhiều lỗ hổng phức tạp vốn chưa được nhận diện trước đó. Các kết quả từ quá trình nghiên cứu, thực nghiệm giúp trả lời cho các câu hỏi RQ1, RQ2 và RQ3.

Trước hết, trả lời cho câu hỏi nghiên cứu RQ1. Trong phiên bản mặc định, thuật toán khám phá của công cụ ZAP hoạt động kém trên ứng dụng Web SPA, các luật kiểm thử chưa nhận diện đầy đủ được các lỗ hổng bảo mật phổ biến thuộc Top 25 CWE. Để khắc phục những hạn chế này, khóa luận đề xuất hai hướng tiếp cận chính: mở rộng khả năng khám phá ứng dụng nhằm giúp ZAP bổ sung đầy đủ URL cho site-map, phát triển các luật kiểm thử tùy chỉnh để nhận diện các lỗ hổng phức tạp mà luật kiểm thử trong phiên bản mặc định chưa bao phủ. Phương pháp triển khai của hai hướng tiếp cận được trình bày cụ thể trong Chương 3.

Tiếp theo, liên quan đến câu hỏi nghiên cứu RQ2, trước khi áp dụng các cải tiến, OWASP ZAP phát hiện hạn chế các lỗ hổng trên các ứng dụng Web mục tiêu, các lỗ hổng phát hiện được liệt kê trong 4.2.3. Sau khi cải tiến, kết hợp phương pháp thủ công và tự động trong giai đoạn xây dựng site-map, cùng với bổ sung các luật kiểm thử tùy chỉnh, giúp ZAP phát hiện được nhiều lỗ hổng hơn, giảm đáng kể tỷ lệ bỏ sót, điều này được thể hiện trong 4.2.4.

Cuối cùng, đối với câu hỏi nghiên cứu RQ3, việc áp dụng kỹ thuật RAG trên LLM hỗ trợ sinh thêm luật kiểm thử giúp người kiểm thử nhanh chóng tiếp cận các lỗ hổng phổ biến và rút ngắn thời gian tạo ra các luật kiểm thử tùy chỉnh. Công

nghe tương tác UI tự động với kịch bản ngôn ngữ tự nhiên *Browser Use* khi tích hợp vào pipeline kiểm thử tự động góp phần giảm thiểu công sức và nhân lực, hạn chế sai sót thủ công, nâng cao hiệu quả xây dựng site-map từ đó mở rộng phạm vi phát hiện lỗ hổng trên các ứng dụng Web SPA. Tuy nhiên, các kỹ thuật LLM triển khai trong khóa luận này mới dừng ở mức minh chứng (PoC); các triển khai tự động hóa tương tác hoặc sinh mã luật kiểm thử vẫn còn hoạt động chậm, thiếu độ chính xác và đôi khi gặp các lỗi liên quan đến cú pháp hoặc sử dụng sai thư viện. Trên thực tế, việc sử dụng LLM Agent tiêu tốn nhiều tài nguyên và chi phí, đồng thời đặt ra yêu cầu tối ưu hóa mô hình và kiểm soát rò rỉ dữ liệu, kịch bản tự động hóa UI cũng dễ bị lỗi khi giao diện ứng dụng thay đổi. Nhìn chung, giải pháp khả thi nhưng cần xem xét kỹ chi phí vận hành, yêu cầu chuyên môn và cơ chế kiểm soát chất lượng để áp dụng hiệu quả trong môi trường thực tế.

Nghiên cứu còn tồn tại một số hạn chế về phạm vi thử nghiệm, tài nguyên triển khai, cũng như khó khăn trong việc đánh giá định tính trên các ứng dụng Web hiện đại. Trong tương lai, các hướng phát triển tiếp theo sẽ tập trung vào việc mở rộng phạm vi áp dụng trên nhiều hệ thống Web thực tế hơn nhằm đưa ra các đánh giá toàn diện và khái quát hơn, đồng thời tối ưu cho trí tuệ nhân tạo trong việc sinh luật, tự động tương tác và khám phá ứng dụng Web. Những nỗ lực tiếp theo hướng tới mục đích hoàn thiện quy trình kiểm thử với công cụ kiểm thử OWASP ZAP, giúp công cụ trở nên toàn diện.

Tài liệu tham khảo

- [1] Verizon. 2025 Data Breach Investigations Report (DBIR). Technical report, Verizon, 2025. [Online]. Available: <https://www.verizon.com/business/resources/reports/dbir>. Accessed: Dec. 01, 2025.
- [2] OWASP Foundation. Source Code Analysis Tools (SAST), 2025. [Online]. Available: https://owasp.org/www-community/Source_Code_Analysis_Tools. Accessed: Dec. 01, 2025.
- [3] OWASP Foundation. Dynamic Application Security Testing (DAST) DevSecOps Guideline, 2025. [Online]. Available: <https://owasp.org/www-project-devsecops-guideline/latest/02b-Dynamic-Application-Security-Testing>. Accessed: Dec. 01, 2025.
- [4] OWASP Foundation. Interactive Application Security Testing (IAST) DevSecOps Guideline, 2025. [Online]. Available: <https://owasp.org/www-project-devsecops-guideline/latest/02c-Interactive-Application-Security-Testing>. Accessed: Dec. 01, 2025.
- [5] Murat Aydos, Çiğdem Aldan, Evren Coşkun, and Alperen Soydan. Security testing of web applications: A systematic mapping of the literature. *Journal of King Saud University-Computer and Information Sciences*, 34(9):6775–6792, 2022.
- [6] E-SPIN. The Importance of Manual Penetration Testing in Web Application Security, 2025. [Online]. Available: <https://www.e-spincorp.com/the-importance-of-manual-penetration-testing-in-web-application-security>. Accessed: Dec. 01, 2025.
- [7] Kuppusamy Vellamadam Palavesam, Siva Venkatesh Arcot, Mahesh Vajinathymala Krishnamoorthy, and GV Ebishdon. Building Automated Security Pipeline for Containerized Microservices. *Journal of Advances in Mathematics and Computer Science*, 40(2):53–66, 2025.
- [8] Francesc Mateo Tudela, Juan-Ramon Bermejo Higuera, Javier Bermejo Higuera, Juan-Antonio Sicilia Montalvo, and Michael I Argy-

- ros. On combining static, dynamic and interactive analysis security testing tools to improve owasp top ten security vulnerability detection in web applications. *Applied Sciences*, 10(24):9119, 2020.
- [9] Edward Chuah, Harsha Kalutarage, Kasim Tasdemir, Atnafu Abrham, and Carsten Maple. A systematic literature review of log-correlation tools for cyberattack detection and prediction in large networks. *Journal of Information Security and Applications*, 92:104096, 2025.
 - [10] Mehmet Ali Yalçinkaya and Ecir Uğur Küçüksille. Artificial Intelligence and Dynamic Analysis-Based Web Application Vulnerability Scanner. *ISeCure*, 16(1), 2024.
 - [11] OWASP Foundation. Zed Attack Proxy (ZAP), 2025. [Online]. Available: <https://www.zaproxy.org>. Accessed: Dec. 01, 2025.
 - [12] PortSwigger. Burp: Web Application Security Testing and Scanning, year = 2025, note = [Online]. Available: <https://portswigger.net/burp>. Accessed: Dec. 01, 2025.
 - [13] OWASP Foundation. OWASP Top 10: 2021, 2021. [Online]. Available: <https://owasp.org/Top10>. Accessed: Dec. 01, 2025.
 - [14] U.S. House of Representatives Committee on Oversight and Government Reform. The Equifax Data Breach, 2018. [Online]. Available: <https://oversight.house.gov/wp-content/uploads/2018/12/Equifax-Report.pdf>. Accessed: Dec. 01, 2025.
 - [15] T. Sellers. Apache Struts Vulnerability (CVE-2017-5638) Exploit Traffic, 2017. [Online]. Available: <https://www.rapid7.com/blog/post/2017/03/09/apache-jakarta-vulnerability-attacks-in-the-wild>. Accessed: Dec. 01, 2025.
 - [16] MITRE. CWE Top 25 (2024), 2024. [Online]. Available: https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html. Accessed: Dec. 01, 2025.
 - [17] MITRE. CWE-90: Improper Neutralization of Special Elements Used in an LDAP Query (LDAP Injection), 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/90.html>. Accessed: Dec. 01, 2025.

- [18] MITRE. CWE-200: Exposure of Sensitive Information to an Unauthorized Actor, 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/200.html>. Accessed: Dec. 01, 2025.
- [19] MITRE. CWE-352: Cross-Site Request Forgery (CSRF), 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/352.html>. Accessed: Dec. 01, 2025.
- [20] MITRE. CWE-93: Improper Neutralization of CRLF Sequences (CRLF Injection), 2024. [Online]. Available: <https://cwe.mitre.org/data/definitions/93.html>. Accessed: Dec. 01, 2025.
- [21] K. Scarfone and P. Mell. Guide to Intrusion Detection and Prevention Systems (IDPS). Technical Report SP 800-94, NIST, 2007. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/94/final>. Accessed: Dec. 01, 2025.
- [22] N. Surribas. Wapiti DAST Tool, 2025. [Online]. Available: <https://wapiti-scanner.github.io>. Accessed: Dec. 01, 2025.
- [23] Project Discovery. Nuclei DAST Tool, 2025. [Online]. Available: <https://github.com/projectdiscovery/nuclei>. Accessed: Dec. 01, 2025.
- [24] Pranjal Kumar. Large language models (LLMs): survey, technical frameworks, and future challenges. *Artificial Intelligence Review*, 57(10):260, 2024.
- [25] Renze Lou, Kai Zhang, and Wenpeng Yin. Large language model instruction following: A survey of progresses and challenges. *Computational Linguistics*, 50(3):1053–1095, 2024.
- [26] Sina Zarrieß, Henrik Voigt, and Simeon Schüz. Decoding methods in neural language generation: A survey. *Information*, 12(9):355, 2021.
- [27] Jie Zhang, Haoyu Bu, Hui Wen, Yongji Liu, Haiqiang Fei, Rongrong Xi, Lun Li, Yun Yang, Hongsong Zhu, and Dan Meng. When llms meet cybersecurity: A systematic literature review. *Cybersecurity*, 8(1):55, 2025.
- [28] Langflow Team. Langflow Documentation, 2025. [Online]. Available: <https://docs.langflow.org>. Accessed: Dec. 01, 2025.
- [29] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim

- Rocktäschel, et al. Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems*, 33:9459–9474, 2020.
- [30] Browser Use. Browser Use: Official Site, 2025. [Online]. Available: <https://browser-use.com>. Accessed: Dec. 01, 2025.
- [31] Oracle. GraalVM: Run Programs Faster Anywhere, 2025. [Online; accessed 06-Dec-2025].
- [32] OWASP Foundation.
- [33] ZAP Community Scripts: A collection of ZAP scripts provided by the community, 2025. [Online; accessed 06-Dec-2025].
- [34] OWASP Foundation. OWASP Juice Shop, 2025. [Online]. Available: <https://owasp.org/www-project-juice-shop>. Accessed: Dec. 01, 2025.
- [35] OWASP Foundation. OWASP Benchmark Project, 2025. [Online]. Available: <https://owasp.org/www-project-benchmark>. Accessed: Dec. 01, 2025.
- [36] NeuraLegion. BrokenCrystal, 2025. [Online]. Available: <https://github.com/NeuraLegion/brokencrystals>. Accessed: Dec. 01, 2025.

XÁC NHẬN CHỈNH SỬA ĐỒ ÁN (KHÓA LUẬN) TỐT NGHIỆP ĐẠI HỌC

Tên tôi là: Lê Khánh Trình

Cơ quan công tác: Trường Đại học Công Nghệ - Đại học Quốc gia Hà Nội

Là cán bộ hướng dẫn đồ án (khóa luận) tốt nghiệp cho:

Sinh viên: Phạm Tất Thành

Sinh ngày: 22/10/2004

Ngành học: Khoa học máy tính

Lớp: QH-2022-I/CQ-I-CS4

Đề tài đồ án (khóa luận) tốt nghiệp: Giải pháp tăng tính hiệu quả của các công cụ kiểm thử bảo mật động cho ứng dụng web.

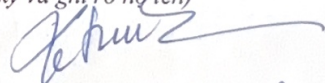
Tôi xác nhận sinh viên Phạm Tất Thành đã chỉnh sửa lại đồ án (khóa luận) tốt nghiệp theo đúng kết luận của Hội đồng chấm đồ án (khóa luận) tốt nghiệp ngày 18/12/2025 gồm các nội dung:

- + Sửa lại đoạn “Đặt vấn đề”
- + Sửa lại trình bày (Hình ảnh có dấu “.” cuối câu, bảng không có dấu “.”)

Hà Nội, ngày 24 tháng 12 năm 2025

KÝ TÊN

(ký và ghi rõ họ tên)


Lê Khánh Trình